



SPECSFY

# Guia completo do usuário

---

Specify. Prove. Ship.

Da primeira ideia à entrega comprovada,  
em uma jornada prática e rastreável.

---

Edição v1.3.0

Gerado em 30 de julho de 2026

Fonte editorial: docs/user/

# Sumário

|                                                      |    |
|------------------------------------------------------|----|
| Guia completo do usuário .....                       | 8  |
| Leia online ou como ebook .....                      | 8  |
| Percurso pedagógico .....                            | 8  |
| Conversa contínua entre etapas .....                 | 10 |
| A ideia central em um exemplo .....                  | 10 |
| Como a metodologia funciona .....                    | 11 |
| Uma única especificação .....                        | 11 |
| Da ideia até o código .....                          | 12 |
| Os três atos .....                                   | 12 |
| Como BDD e TDD trabalham juntos .....                | 14 |
| O agente conduz as transições .....                  | 15 |
| Mudanças durante o trabalho .....                    | 15 |
| Informações que permanecem entre entregas .....      | 15 |
| O que você encontra ao final .....                   | 16 |
| Limites do método .....                              | 16 |
| Justificativa de tamanho .....                       | 16 |
| Instalação do Specsify .....                         | 17 |
| Instale o CLI .....                                  | 17 |
| Prepare o projeto consumidor .....                   | 17 |
| Confira os arquivos instalados .....                 | 18 |
| Atualize sem perder customizações .....              | 18 |
| Corrija falhas comuns .....                          | 19 |
| Primeiro projeto com o Specsify .....                | 20 |
| O que você vai construir .....                       | 20 |
| Capture uma entrada .....                            | 20 |
| Refine a proposta no backlog .....                   | 20 |
| Crie a especificação única .....                     | 21 |
| Comprove a definição .....                           | 22 |
| Organize as tarefas .....                            | 22 |
| Prove que o teste detecta a ausência da página ..... | 22 |
| Implemente e valide .....                            | 23 |
| Incorpore uma mudança posterior .....                | 23 |
| Consulte o estado final .....                        | 24 |

|                                                               |    |
|---------------------------------------------------------------|----|
| Continue na mesma conversa .....                              | 24 |
| Justificativa de tamanho .....                                | 24 |
| Manutenção deste guia .....                                   | 24 |
| Inbox .....                                                   | 25 |
| Capturar .....                                                | 25 |
| O que o arquivo organiza .....                                | 25 |
| Inbox, backlog e spec .....                                   | 26 |
| Templates instalados .....                                    | 26 |
| Refinar ideias no backlog .....                               | 27 |
| Estrutura .....                                               | 27 |
| Organização do backlog .....                                  | 27 |
| Anatomia de um item .....                                     | 28 |
| Padrões recorrentes .....                                     | 29 |
| Ordenar o backlog .....                                       | 30 |
| Fluxo .....                                                   | 31 |
| Estados do backlog .....                                      | 31 |
| Quando está refinado .....                                    | 31 |
| Limites .....                                                 | 32 |
| Próximo passo .....                                           | 32 |
| Skills base .....                                             | 33 |
| Encontre a skill pelo estado do trabalho .....                | 33 |
| Capturar uma entrada com <code>specsfy-01-inbox</code> .....  | 35 |
| Quando usar .....                                             | 35 |
| Como descrever a tarefa .....                                 | 35 |
| Exemplo passo a passo .....                                   | 35 |
| O que esperar .....                                           | 35 |
| Erros comuns .....                                            | 36 |
| Próximo passo .....                                           | 36 |
| Refinar uma entrada com <code>specsfy-02-backlog</code> ..... | 37 |
| Quando usar .....                                             | 37 |
| Como descrever a tarefa .....                                 | 37 |
| Exemplo passo a passo .....                                   | 37 |
| Como o ciclo termina .....                                    | 38 |
| O que esperar .....                                           | 38 |
| Erros comuns .....                                            | 38 |

|                                                                  |    |
|------------------------------------------------------------------|----|
| Próximo passo .....                                              | 39 |
| Criar a especificação com <code>specsify-03-specify</code> ..... | 40 |
| Quando usar .....                                                | 40 |
| Como descrever a tarefa .....                                    | 40 |
| Exemplo passo a passo.....                                       | 40 |
| O que esperar .....                                              | 41 |
| Erros comuns.....                                                | 41 |
| Próximo passo .....                                              | 41 |
| Validar a definição com <code>specsify-04-validate</code> .....  | 42 |
| Quando usar .....                                                | 42 |
| Como descrever a tarefa .....                                    | 42 |
| Exemplo passo a passo.....                                       | 42 |
| O que esperar .....                                              | 42 |
| Erros comuns.....                                                | 43 |
| Próximo passo .....                                              | 43 |
| Planejar tarefas com <code>specsify-05-tasks</code> .....        | 44 |
| Quando usar .....                                                | 44 |
| Como descrever a tarefa .....                                    | 44 |
| Exemplo passo a passo.....                                       | 44 |
| O que esperar .....                                              | 44 |
| Erros comuns.....                                                | 45 |
| Próximo passo .....                                              | 45 |
| Preparar testes com <code>specsify-06-tdd-bdd</code> .....       | 46 |
| Quando usar .....                                                | 46 |
| Como descrever a tarefa .....                                    | 46 |
| Exemplo passo a passo.....                                       | 46 |
| O que esperar .....                                              | 47 |
| Erros comuns.....                                                | 47 |
| Próximo passo .....                                              | 47 |
| Entregar código com <code>specsify-07-implement</code> .....     | 48 |
| Quando usar .....                                                | 48 |
| Como descrever a tarefa .....                                    | 48 |
| Exemplo passo a passo.....                                       | 48 |
| O que esperar .....                                              | 48 |
| Erros comuns.....                                                | 49 |

|                                                                |    |
|----------------------------------------------------------------|----|
| Próximo passo .....                                            | 49 |
| Incorporar mudanças com <code>specsfy-update-spec</code> ..... | 50 |
| Quando usar .....                                              | 50 |
| Como descrever a tarefa .....                                  | 50 |
| Exemplo passo a passo .....                                    | 50 |
| O que esperar .....                                            | 51 |
| Erros comuns .....                                             | 51 |
| Próximo passo .....                                            | 51 |
| Consultar o estado com <code>specsfy-progress</code> .....     | 52 |
| Quando usar .....                                              | 52 |
| Como descrever a tarefa .....                                  | 52 |
| Exemplo passo a passo .....                                    | 52 |
| O que esperar .....                                            | 52 |
| Erros comuns .....                                             | 53 |
| Próximo passo .....                                            | 53 |
| CLI e TUI do Specsify .....                                    | 54 |
| Instalar e gerenciar skills .....                              | 54 |
| Atualização automática .....                                   | 55 |
| Dashboard e progresso .....                                    | 55 |
| Testes do projeto .....                                        | 59 |
| Justificativa de tamanho .....                                 | 61 |
| Segurança e reversibilidade .....                              | 61 |
| Informações permanentes do projeto .....                       | 63 |
| Monitoramento durante mudanças .....                           | 64 |
| Preservação e atualização .....                                | 64 |
| Documentação técnica do sistema .....                          | 66 |
| Atualizar uma especificação .....                              | 68 |
| Descreva o que mudou .....                                     | 68 |
| O que acontece .....                                           | 68 |
| Resultado esperado .....                                       | 69 |
| Limites .....                                                  | 69 |
| Quando usar outra skill .....                                  | 69 |
| Skills especialistas .....                                     | 70 |
| Detectar e instalar .....                                      | 70 |
| Catálogo por domínio .....                                     | 70 |

|                                            |    |
|--------------------------------------------|----|
| Relação com as bases .....                 | 71 |
| Uso avançado do Specsify .....             | 72 |
| Detecte e instale orientação técnica ..... | 72 |
| Automatize a leitura de progresso .....    | 72 |
| Ajuste a atualização visual .....          | 73 |
| Atualize sem perder customizações .....    | 73 |
| Incorpore uma mudança na spec .....        | 73 |
| Limites .....                              | 74 |
| Usar Specsify com Laravel .....            | 75 |
| Confirmar a detecção .....                 | 75 |
| Instalação .....                           | 75 |
| Aplicar na spec .....                      | 75 |
| O que o especialista acrescenta .....      | 76 |
| Resultado esperado .....                   | 76 |
| Limites .....                              | 76 |
| Usar Specsify com Astro .....              | 77 |
| Confirmar a detecção .....                 | 77 |
| Instalação .....                           | 77 |
| Aplicar na spec .....                      | 77 |
| O que o especialista acrescenta .....      | 78 |
| Resultado esperado .....                   | 78 |
| Limites .....                              | 78 |
| Usar Specsify com Next.js .....            | 79 |
| Confirmar a detecção .....                 | 79 |
| Instalação .....                           | 79 |
| Aplicar na spec .....                      | 79 |
| O que o especialista acrescenta .....      | 80 |
| Resultado esperado .....                   | 80 |
| Limites .....                              | 80 |
| Créditos do Specsify .....                 | 81 |
| Projeto e manutenção .....                 | 81 |
| Comunidade .....                           | 81 |
| Identidade .....                           | 81 |
| Componentes relacionados .....             | 81 |
| Inspirações e fontes .....                 | 81 |

|                       |    |
|-----------------------|----|
| Como contribuir ..... | 82 |
|-----------------------|----|

# Guia completo do usuário

---



O Specsify ajuda você a transformar uma ideia em software testado sem espalhar requisitos, planos e tarefas por vários arquivos. Você conversa normalmente com o agente, e as skills organizam o trabalho em uma única especificação. Nesse arquivo, você consegue conferir o que será entregue, quais testes comprovam o comportamento e o que já foi concluído.

Este guia começa pela lógica da metodologia, prepara o ambiente e acompanha uma entrega completa. Os capítulos seguintes explicam a rotina com o CLI, as mudanças posteriores e os recursos avançados. Você não precisa conhecer a implementação do framework para seguir esse percurso.

## Leia online ou como ebook

Este mesmo percurso compõe a edição portátil **v1.2.5**. Use o PDF para leitura e impressão ou o EPUB em leitores que permitem ajustar fonte e tamanho:

- PDF, para leitura, compartilhamento e impressão.
- EPUB, para leitores digitais com fonte e tamanho ajustáveis.

Os dois formatos são reconstruídos a partir destas páginas. A [pasta do ebook](#) informa a versão vigente e os hashes usados para conferir se o PDF e o EPUB correspondem ao mesmo build.

## Percurso pedagógico

Siga a ordem abaixo na primeira leitura. Quando já conhecer o método, use os links para voltar diretamente à tarefa que precisa executar.

### 1. Entenda a metodologia

Comece pela [Metodologia](#). Cada entrega mantém o problema, os requisitos, os exemplos de comportamento, o plano técnico, as tarefas, os testes e as evidências em uma única `spec.md`. O capítulo mostra como esse arquivo muda ao longo do trabalho e o que comprova a passagem entre os atos.

Os três atos ligam cada fase a uma evidência verificável na `spec.md` :

1. **Ato I — Definir:** entender e validar o que deve ser entregue.
2. **Ato II — Projetar e provar:** preparar tarefas e obter o RED, a falha esperada antes da implementação.
3. **Ato III — Entregar e validar:** implementar, obter testes verdes e registrar evidências.

## 2. Instale o Specsify

Com o método entendido, siga a [Instalação](#) para instalar o CLI e preparar seu repositório. Ao final, `specsify skills list` mostra as skills disponíveis e `specsify progress --project .` confirma que o CLI consegue ler o projeto, mesmo que ainda não exista uma spec.

## 3. Faça a primeira entrega

Use [Primeiro projeto](#) como tutorial guiado. Você começa com uma mudança pequena, acompanha a criação da `spec.md` e termina conferindo os testes e as evidências registradas, sem precisar decorar cada skill.

Para preservar uma ideia sem iniciar a especificação, escolha uma destas entradas:

- preserve um texto sem perguntas na [Inbox](#).
- refine e priorize uma proposta no [Backlog](#).

## 4. Aprofunde o fluxo base

O índice de [Skills base](#) apresenta o fluxo completo. Leia cada etapa nesta ordem:

1. [Capturar uma entrada](#).
2. [Refinar no backlog](#).
3. [Criar a especificação](#).
4. [Validar a definição](#).
5. [Preparar as tarefas](#).
6. [Preparar TDD e BDD](#).
7. [Implementar](#).
8. [Atualizar a especificação](#).
9. [Consultar o progresso](#).

Essas páginas explicam quando usar cada skill, como descrever a tarefa em linguagem natural, o resultado esperado, os erros comuns e o próximo passo.

## 5. Opere o projeto no dia a dia

Depois da primeira entrega, escolha os guias ligados à sua rotina:

- [CLI e TUI](#): comandos, interface visual e acompanhamento.
- [Informações permanentes do projeto](#): stack, regras, banco e convenções.
- [Documentação do sistema](#): documentação técnica derivada da aplicação.
- [Mudanças posteriores](#): como incorporar um novo requisito à mesma especificação.

## 6. Avance quando precisar

Os próximos guias são opcionais. Consulte-os quando a entrega exigir uma integração ou tecnologia específica:

- [Especialistas](#), para conhecimento técnico adicional.
- [Uso avançado](#), para automação e integrações.
- aplicação em projetos [Laravel](#), [Astro](#) ou [Next.js](#).
- Quadro técnico, para conhecer os módulos do monorepo.
- [Créditos](#), para autoria e identidade do projeto.

Se você pretende contribuir ou modificar o próprio framework, continue no [guia técnico](#). Ele é um percurso separado do uso em projetos consumidores.

## Conversa contínua entre etapas

Quando uma etapa depende de outra skill, o agente anuncia a transição, explica o que falta e retoma o trabalho na mesma conversa. Você acompanha a mudança de etapa sem repetir a instrução inicial nem escolher manualmente cada skill.

## A ideia central em um exemplo

Imagine uma página de boas-vindas. Você pode preservar a ideia, refiná-la no backlog e promovê-la até chegar a:

```
specs/specs/0001-pagina-boas-vindas/spec.md
```

Em seguida, o agente valida a definição, organiza tarefas, prepara testes, implementa e registra evidências nesse mesmo arquivo. Se depois você solicitar um botão novo, a alteração retorna à mesma `spec.md`. O agente reabre somente os atos cujas provas perderam validade, sem criar `plan.md`, `tasks.md` ou outra fonte normativa.

Para começar esse percurso com orientação passo a passo, siga agora [a Metodologia](#).

# Como a metodologia funciona

---

O Specsify ajuda você a transformar uma necessidade em uma entrega comprovada. Você explica o que precisa, e o agente organiza a definição, o plano, os testes e a implementação na mesma especificação.

A metodologia existe para responder, durante todo o trabalho, a três perguntas:

1. **O que será entregue?**
2. **O que comprova que o plano está pronto?**
3. **Qual evidência comprova que a entrega funciona?**

Você não precisa decorar comandos nem escolher cada skill. O agente identifica a etapa atual, anuncia as transições e mantém o trabalho na mesma conversa.

## Uma única especificação

Cada mudança escolhida possui uma única fonte normativa. O caminho permite que você reconheça a sequência e o assunto ao listar o diretório de specs:

```
specs/specs/<NNNN>-<slug>/spec.md
```

NNNN é o número da spec, como 0001. O slug identifica o assunto, como recuperar-senha. Assim, o caminho 0001-recuperar-senha pode ser localizado sem abrir o arquivo.

Ao abrir a spec.md, você encontra o comportamento esperado, o plano e as provas que permitem conferir o estado atual:

- o problema e o resultado esperado.
- as pessoas afetadas e as regras que precisam ser respeitadas.
- histórias, requisitos e limites.
- exemplos de comportamento escritos em BDD.
- as escolhas registradas, as possíveis falhas e o plano técnico.
- tarefas, testes e evidências da execução.
- gates e estado atual da entrega.

O plano e as tarefas ficam dentro da própria spec. O Specsify não cria plan.md ou tasks.md, então a explicação da entrega não se divide entre arquivos com estados diferentes.

## Da ideia até o código

Nem todo texto precisa virar uma entrega imediatamente. Você escolhe o destino de acordo com o quanto já definiu:

- Uma **ideia** preserva seu texto em `specs/inbox/` sem perguntas e sem autorizar implementação.
- O **backlog** guarda uma proposta que merece organização e refinamento, mas ainda não foi escolhida para entrega. Ela vive em `specs/backlog/`.
- A **spec** representa uma entrega escolhida. A partir dela, definição, plano, testes, código e evidências passam a seguir o mesmo contrato.

O percurso mais completo preserva a ideia, aprofunda as definições e só chega ao código depois do plano e do RED:

```
inbox → backlog → spec → plano e RED → código → validação
```

Você também pode começar com “implemente recuperação de senha”. O agente só altera o código depois de verificar as definições ausentes e conduzir as etapas correspondentes.

## Os três atos

Os atos são grupos de trabalho. Cada um termina com um **gate**, que é um ponto de controle baseado em evidência. Um **gate** aprovado não significa apenas “parece bom”: significa que as condições daquela etapa foram verificadas.

### Escolha o destino da ideia

**Objetivo:** preservar a ideia sem transformá-la imediatamente em spec.

**Sua participação:** você informa a ideia e escolhe quando vale refiná-la ou promovê-la. Se solicitar apenas a captura, o agente não inicia o refinamento.

**Prova técnica:** a captura recebe um arquivo próprio em `specs/inbox/`. Se você escolher o refinamento, ela segue para o backlog. A `spec.md` normativa só aparece depois de uma promoção explícita.

Essa separação mantém a caixa de entrada leve e impede que toda observação se transforme em código ou em uma especificação extensa.

### Ato I — Definir o que precisa mudar

**Objetivo:** entender o problema e transformar a intenção em comportamento que possa ser conferido.

**Sua participação:** você responde apenas às dúvidas que realmente mudam a entrega. O agente reaproveita o que já foi informado, pergunta sobre uma lacuna importante por vez e não inventa requisitos quando falta uma definição. O ciclo segue sem limite máximo de perguntas: cada resposta atualiza a análise, e uma nova pergunta é feita enquanto restar uma lacuna aplicável. A partir da 11ª pergunta, você também pode escolher `avanzar`; as lacunas ficam registradas e o Definition Gate continua pendente até serem resolvidas.

**Prova técnica:** a spec registra a finalidade, as pessoas afetadas, os requisitos, os limites e os cenários BDD. A validação procura contradições, definições em aberto e dúvidas que impedem o planejamento.

O Ato I termina quando a validação registra este gate na `spec.md`:

```
Definition Gate: Passed
```

Esse estado permite que o agente organize o plano a partir dos requisitos e cenários já conferidos. O código ainda não foi alterado, e o Plan Gate continua pendente.

## Ato II — Planejar e preparar o RED

**Objetivo:** definir como a mudança será construída e demonstrar que os testes conseguem detectar a ausência do novo comportamento.

**Sua participação:** você confirma escolhas de produto ou estratégia que alteram o resultado. Os detalhes técnicos que o repositório consegue comprovar podem ser derivados do código, da stack e das regras do projeto.

**Prova técnica:** o agente registra as tarefas, os contratos, as informações afetadas, as possíveis falhas e a reversibilidade na spec. Depois, transforma os cenários BDD em testes TDD executáveis e observa o **RED**, uma falha causada pela funcionalidade que ainda não existe.

O Ato II termina quando as tarefas e os testes permitem registrar:

```
Plan Gate: Passed
```

O RED precisa falhar pelo motivo esperado. Erro de sintaxe, dependência ausente ou ambiente quebrado não prova que o teste protege o comportamento.

## Ato III — Entregar e conferir o resultado

**Objetivo:** implementar cada tarefa e reunir evidências atuais de que o resultado atende à definição.

**Sua participação:** você acompanha novas escolhas ou mudanças de escopo. O agente registra cada comando executado e apresenta o resultado verificável, sem exigir que você conduza os ciclos de teste.

**Prova técnica:** cada tarefa de código parte de um teste que falha pelo motivo esperado, recebe a menor implementação capaz de deixá-lo verde e termina com refatoração protegida pela suíte:

```
RED → GREEN → REFACTOR
```

- **RED:** o teste falha pela razão esperada.
- **GREEN:** a menor implementação faz o teste passar.
- **REFACTOR:** o código é melhorado sem mudar o comportamento.

Depois, o agente executa o aceite e a regressão completa, verifica a ligação entre cada requisito e seu teste, atualiza os registros permanentes do projeto e reconstrói a documentação aplicável.

O Ato III termina quando o aceite, a regressão e a documentação permitem registrar:

```
Delivery Gate: Passed  
Status: Complete
```

**Complete** significa que a entrega possui código e evidência atual. Você pode abrir a spec e localizar os comandos, os resultados e os testes que comprovam esse estado.

## Como BDD e TDD trabalham juntos

O BDD (desenvolvimento orientado por comportamento) descreve o resultado que produto e desenvolvimento precisam discutir. O TDD (desenvolvimento orientado por testes) transforma esse comportamento em uma prova executável no projeto.

O **BDD** descreve o comportamento em uma linguagem que produto, desenvolvimento e testes conseguem discutir. Por exemplo:

```
Cenário: cliente solicita recuperação de senha  
  Dado que existe um cadastro para o e-mail informado  
  Quando o cliente solicita a recuperação  
  Então o sistema confirma a solicitação sem revelar informações privadas
```

Esse cenário mostra a regra e o resultado esperado, mas o texto Gherkin não é executado como uma suíte separada pelo Specsify.

O **TDD** transforma o comportamento em testes executáveis na ferramenta já usada pelo projeto. Primeiro o teste falha pelo motivo correto. Depois, a implementação faz o teste passar. Em resumo:

- BDD ajuda a definir **o comportamento que importa**.

- TDD comprova **que o código apresenta esse comportamento**.

Essa ligação evita uma descrição clara sem prova automática e também impede que um teste técnico seja aceito sem representar a necessidade registrada.

## O agente conduz as transições

As skills dividem responsabilidades, mas você não precisa operar o fluxo como uma lista de comandos. Quando uma etapa depende de outra, o agente:

1. informa de qual skill está saindo e para qual está indo.
2. explica a pendência que motivou a transição.
3. resolve a pendência na skill responsável.
4. retorna à etapa anterior quando necessário.

Por exemplo, se a implementação encontrar um teste ausente, o agente volta ao planejamento e à preparação TDD, obtém um RED válido e só então retoma o código. Ele não aprova um gate apenas para contornar a pendência.

## Mudanças durante o trabalho

Se a necessidade mudar depois que a spec já existe, explique a alteração em linguagem normal. O novo requisito entra na mesma `spec.md`, sem criar uma segunda especificação.

O Specsify reabre somente as provas que perderam validade:

- Uma mudança de comportamento reabre definição, plano e entrega.
- Uma mudança apenas na estratégia técnica reabre plano e entrega.
- Uma evidência desatualizada exige somente a repetição da validação correspondente.

Use [specsify-update-spec](#) para incorporar a nova instrução. A skill informa quais gates perderam validade e retoma a primeira etapa afetada.

## Informações que permanecem entre entregas

Além da spec de cada entrega, o projeto mantém informações que valem para o sistema inteiro:

- `PROJECT.md` : finalidade, capacidades e limites do projeto.
- `.specsify/STACK.md` : tecnologias estruturais e suas evidências.
- `.specsify/RULES.md` : regras confirmadas para o trabalho.
- `.specsify/DATABASE.md` : visão da persistência e das relações.

O agente consulta esses arquivos antes de planejar e os revisa durante a implementação. Assim, uma nova entrega começa com a arquitetura, as convenções e o banco já documentados.

## O que você encontra ao final

Uma mudança completa deixa na `spec.md` um caminho auditável entre requisito, teste, tarefa e evidência:

- a intenção e as escolhas estão na spec.
- cada requisito aponta para condições de aceite.
- os cenários BDD explicam o comportamento.
- os testes TDD demonstram o resultado no código.
- as tarefas registram execução e evidências.
- os gates mostram quais etapas foram realmente comprovadas.
- a documentação reflete o sistema implementado.

Para consultar esse estado sem alterar arquivos, use [specsfy-progress](#).

## Limites do método

O método não define requisitos importantes sem você, não transforma toda ideia em spec e não trata pesquisa como requisito aprovado. Também não aceita erro de ambiente como RED nem substitui os testes e as ferramentas do seu projeto.

## Justificativa de tamanho

Este guia mantém os três atos, os gates e a relação entre BDD e TDD na mesma página para que você possa comparar o percurso completo sem alternar entre explicações parciais.

O [guia de instalação](#) prepara o CLI e o framework. Depois, o [primeiro projeto](#) aplica os três atos a uma página de boas-vindas e mostra os gates na `spec.md`.

# Instalação do Specsfy

---

## Instale o CLI

O Specsfy requer Node.js 22.12 ou uma versão mais recente. O pacote oficial publicado no npm instala o comando no ambiente global do usuário:

```
npm install --global @promovaweb/specsfy
```

Execute `specsfy --version` para confirmar que o terminal localiza o comando e que o Node.js consegue abrir o aplicativo:

```
specsfy --version
```

Uma resposta com o número da versão confirma a instalação. Se o terminal mostrar `specsfy: command not found`, consulte o diretório global do npm e confirme se o diretório de executáveis está no `PATH`:

```
npm prefix --global  
specsfy --version
```

O download em `get.specsfy.dev` continua disponível para instalações mantidas em `$HOME/.local/bin`. Esse executável inclui as dependências do CLI, mas também requer Node.js 22.12 ou superior:

```
mkdir -p "$HOME/.local/bin"  
curl -fL get.specsfy.dev -o "$HOME/.local/bin/specsfy"  
chmod +x "$HOME/.local/bin/specsfy"
```

## Prepare o projeto consumidor

Abra a raiz do repositório que receberá a metodologia. Não execute a instalação dentro do monorepo oficial do Specsfy, porque o CLI reconhece essa raiz como ambiente de desenvolvimento e recusa a operação:

```
cd caminho/do/projeto  
specsfy install --project .
```

O instalador publica as etapas numeradas a partir de `.agents/skills/specsfy-01-inbox`, grava o contrato central em `.specsfy/Spec.md` e adiciona templates, exemplos e registros técnicos em `.specsfy/`. Ele também insere blocos gerenciados em `AGENTS.md` e `CLAUDE.md`, preservando o conteúdo que já existe fora desses blocos.

Para personalizar um template sem impedir atualizações, copie-o para `.specsfy/templates/custom/` com o mesmo nome. Essa versão tem precedência e nunca é sobrescrita pelo instalador, inclusive com `--force`.

A instalação inclui as nove skills base, o setup, o documentador do sistema e as três skills auxiliares. Ela prepara os arquivos usados pelo agente, mas não cria uma spec de produto nem altera o código da aplicação.

## Confira os arquivos instalados

Na mesma raiz, liste o catálogo e consulte o progresso. O primeiro comando deve mostrar as skills instaladas, e o segundo deve conseguir ler o diretório de specs:

```
specsfy skills list
specsfy progress --project .
```

O catálogo deve mostrar as skills do Specsify. Em um projeto novo, o comando de progresso pode retornar zero specs. Esse resultado confirma que o CLI leu o repositório e ainda não encontrou arquivos em `specs/specs/<NNNN>-<slug>/spec.md`.

O comando sem subcomando abre a interface visual no diretório atual. O nome do projeto aparece no topo e as abas devem carregar mesmo quando ainda não houver spec:

```
specsfy
```

O dashboard deve carregar as abas do projeto mesmo quando as tabelas ainda estiverem vazias. Use `Ctrl+Q` para sair.

## Atualize sem perder customizações

Para atualizar o pacote instalado pelo npm:

```
npm update --global @promovaweb/specsfy
specsfy --version
```

Na instalação pelo arquivo de `get.specsfy.dev`, repita o download e a permissão de execução. Para atualizar as skills já instaladas, execute `skills update`. O CLI compara os fingerprints e preserva os arquivos customizados:

```
specsify skills update --project .
```

O Specsify registra fingerprints dos arquivos gerenciados. Uma atualização normal substitui versões intactas e preserva arquivos customizados. Se o CLI informar que encontrou alterações locais, revise a diferença. `--force` descarta a customização protegida no arquivo indicado.

## Corrija falhas comuns

- **Comando ausente:** confira o resultado de `npm prefix --global` e o `PATH` usado pelo terminal.
- **Node.js incompatível:** execute `node --version`. O aplicativo requer Node.js 22.12 ou uma versão mais recente.
- **Permissão negada:** execute novamente `chmod +x "$HOME/.local/bin/specsify"` quando usar o download. Em instalações pelo npm, configure um diretório global gravável pelo seu usuário.
- **Instalação das skills interrompida:** disponibilize o comando `skills` ou o `npx`, usado pelo CLI como alternativa.
- **Arquivo gerenciado customizado:** preserve sua versão ou compare as mudanças oficiais e só então repita o comando com `--force`.

Com o ambiente conferido, siga o [primeiro projeto](#) para criar uma entrega pequena e observar a primeira `spec.md`. O [guia do CLI e da TUI](#) detalha os comandos de atualização, progresso, testes e configuração.

# Primeiro projeto com o Specsify

---

## O que você vai construir

Este tutorial acompanha uma página de boas-vindas em um projeto Laravel que já usa Pest. A rota recebe um nome e mostra uma saudação. Quando o nome não for informado, a página usa `visitante`.

Você verá como uma ideia chega à implementação sem dividir a fonte normativa entre `plan.md`, `tasks.md` e outros arquivos. O exemplo mostra cada skill separadamente para facilitar a consulta, embora o agente consiga fazer as transições na mesma conversa.

O tutorial depende de três condições verificáveis. Confirme a instalação, abra o agente na raiz do projeto consumidor e rode a suíte Pest existente:

- o CLI e o framework foram instalados conforme o [guia de instalação](#).
- o agente está aberto na raiz do projeto consumidor.
- o repositório possui um runner Pest funcional.

## Capture uma entrada

Use `$specsify-01-inbox` para guardar a formulação original em `specs/inbox/`. A captura não inicia perguntas nem altera o código:

```
Use $specsify-01-inbox para capturar:  
criar uma página /boas-vindas que cumprimente o visitante pelo nome.
```

A skill grava um arquivo com data, horário e slug em `specs/inbox/`. O relato deve apontar um caminho semelhante a este:

```
specs/inbox/2026-07-28-143205-pagina-boas-vindas.md
```

Essa captura preserva o texto recebido e registra inferências separadamente. Ela ainda não cria backlog, spec, tarefas ou código.

## Refine a proposta no backlog

Quando a ideia merecer refinamento, envie o arquivo para `$specsify-02-backlog`. A skill lê a captura preservada, procura relações e cria um item numerado:

```
Use $specsfy-02-backlog para refinar
specs/inbox/2026-07-28-143205-pagina-boas-vindas.md
```

Você também pode fornecer o texto diretamente. A skill procura material relacionado, esclarece somente o necessário para o backlog e grava um item numerado:

```
specs/backlog/0001-pagina-boas-vindas.md
```

O backlog organiza uma possibilidade de entrega, mas não autoriza alteração no código. Essa separação permite comparar e priorizar ideias antes de criar uma especificação normativa.

Use `$specsfy-02-backlog` para aprofundar o item. A conversa pergunta uma lacuna aplicável por vez e retorna um brief:

```
Use $specsfy-02-backlog em
specs/backlog/0001-pagina-boas-vindas.md
```

O agente reaproveita o conteúdo existente e pergunta uma lacuna relevante por vez. Neste exemplo, a resposta padrão muda o comportamento visível da página:

```
Agente: O que deve aparecer quando nenhum nome for informado?
Você: Olá, visitante!
```

O refinamento do backlog produz um brief na conversa. Por padrão, ele não cria uma segunda fonte normativa nem modifica o backlog.

## Crie a especificação única

Depois de resolver as dúvidas materiais, promova o backlog com `$specsfy-03-specify`:

```
Use $specsfy-03-specify para promover
specs/backlog/0001-pagina-boas-vindas.md
```

A skill cria o diretório numerado e mantém a fonte normativa neste caminho:

```
specs/specs/0001-pagina-boas-vindas/spec.md
```

Abra esse arquivo e confira se o problema, as pessoas afetadas, os requisitos, os limites e os cenários BDD representam a conversa. O Gherkin permanece na spec como referência legível. O Specsfy não cria uma suíte `.feature` separada.

## Comprove a definição

Use `$specsify-04-validate` para auditar a spec. A skill informa a localização de cada falha e só aprova o Definition Gate quando a definição estiver completa:

```
Use $specsify-04-validate em
specs/specs/0001-pagina-boas-vindas/spec.md
```

Uma definição pronta termina a validação com estes dois sinais:

```
READY
Definition Gate: Passed
```

`READY` confirma que a spec possui as informações necessárias para planejar. O estado ainda não afirma que a página existe. Se houver contradição ou uma escolha importante em aberto, a validação retorna à skill responsável antes de aprovar o gate.

## Organize as tarefas

Use `$specsify-05-tasks` para manter o plano e as tarefas dentro da mesma `spec.md`. O arquivo deve mostrar os requisitos cobertos e a dependência entre o teste em RED e cada tarefa de produção:

```
Use $specsify-05-tasks em
specs/specs/0001-pagina-boas-vindas/spec.md
```

A skill separa testes, código, documentação e trabalho operacional, registra dependências e liga cada tarefa aos requisitos correspondentes. Ela não cria `tasks.md` nem altera o código de produção.

## Prove que o teste detecta a ausência da página

Use `$specsify-06-tdd-bdd` no modo de preparação:

```
Use $specsify-06-tdd-bdd em
specs/specs/0001-pagina-boas-vindas/spec.md para preparar o TDD.
```

Como o projeto do exemplo usa PHP, a skill cria testes Pest derivados dos cenários BDD. Cada caso executável recebe seu marcador `SPECSFY:` junto à definição. A feature inteira e cada história ou requisito aplicável precisam ter, no mínimo, três casos distintos: caminho feliz, variação importante e falha ou limite material.

Execute o teste focal e confirme o RED pelo motivo esperado. Uma rota ausente prova que o teste detecta o comportamento ainda não implementado. Erro de sintaxe, fixture quebrada ou dependência ausente precisa ser corrigido antes de o RED ser aceito. Depois que as tarefas e seus predecessores TDD estiverem coerentes, o `Plan Gate` pode chegar a `Passed`.

## Implemente e valide

Com os gates de definição e plano aprovados, use `$specsify-07-implement`:

```
Use $specsify-07-implement em
specs/specs/0001-pagina-boas-vindas/spec.md
```

A implementação percorre cada tarefa em `RED → GREEN → REFACTOR`. Para uma tarefa de código, a skill exige um predecessor TDD com RED registrado, cria a menor mudança capaz de deixar o teste verde e executa a regressão aplicável. Os comandos, os resultados e os IDs cobertos entram como evidência na spec.

Depois de cada tarefa de código, o agente chama `$specsify-documentator`. Essa skill reconstrói a documentação técnica em `docs/` a partir do sistema existente e executa o modo `--check`. O fluxo só retoma a implementação quando a documentação representar o código atual.

No fechamento, a implementação verifica aceite, regressão, rastreabilidade, documentação e Definition of Done. Uma entrega comprovada termina com:

```
Delivery Gate: Passed
Status: Complete
```

## Incorpore uma mudança posterior

Imagine que, depois da primeira entrega, o nome precise aceitar no máximo 80 caracteres. Use `$specsify-update-spec` na spec existente:

```
Use $specsify-update-spec em
specs/specs/0001-pagina-boas-vindas/spec.md:
o nome deve ter no máximo 80 caracteres.
```

Essa skill preserva a nova instrução, atualiza a `spec.md` e invalida somente as provas afetadas. Como o limite muda comportamento, o fluxo reabre desde o Ato I, percorre validação, tarefas e TDD/BDD, e só então retoma `$specsify-07-implement`. A skill de atualização não altera código de produção automaticamente.

Uma alteração restrita ao plano técnico reabre os Atos II e III. Uma correção editorial comprovadamente sem mudança de significado preserva os gates. Em todos os casos, o histórico continua na mesma spec.

## Consulte o estado final

Use `$specsify-progress` para projetar o estado sem editar arquivos:

```
Use $specsify-progress para mostrar o resultado final.
```

O relatório mostra specs, gates, tarefas, checklists, pendências e o próximo trabalho disponível. Você também pode consultar o mesmo estado pelo CLI:

```
specsify progress --project .  
specsify progress --project . --json  
specsify tui --project .
```

Uma entrega pronta deve aparecer como `Complete`, com os três gates aprovados e sem pendência documental. Capturas em `specs/inbox/` e itens em `specs/backlog/` não entram nesse cálculo.

## Continue na mesma conversa

Você não precisa enviar cada exemplo deste tutorial manualmente. Ao autorizar a jornada completa, uma skill anuncia o handoff, carrega a próxima responsabilidade e retoma a etapa anterior quando necessário. A transição automática não amplia permissões para deploy, publicação, instalação de especialista ou ação destrutiva.

Agora aprofunde os [comandos do CLI e da TUI](#), consulte as [informações permanentes do projeto](#) ou conheça o [uso avançado](#).

## Justificativa de tamanho

O tutorial acompanha uma única entrega desde a captura até o estado `Complete`. Manter o exemplo em uma página permite conferir como cada arquivo e gate é produzido pelo resultado da etapa anterior.

## Manutenção deste guia

Atualize esta página quando a sequência dos atos, a responsabilidade de uma skill base, os gates, os estados ou os caminhos canônicos mudarem. Use a metodologia executável em `skills/` como fonte e preserve `specs/specs/<NNNN>-<slug>/spec.md` como a única fonte normativa de cada fatia no projeto consumidor.

# Inbox

---

`specs/inbox/` recebe entradas antes de qualquer refinamento ou definição de produto. A captura é deliberadamente rápida. O agente guarda o texto sem fazer perguntas e sem transformá-lo em compromisso de entrega.

## Capturar

Use `$specsfy-01-inbox` para guardar esta entrada:  
quero permitir que a pessoa continue um formulário em outro dispositivo.

O agente grava a captura em `specs/inbox/` com data, hora e um nome derivado do conteúdo:

`specs/inbox/AAAA-MM-DD-HHMMSS-<slug>.md`

Data e hora evitam uma fila numerada artificial e preservam a ordem real de chegada. Se duas capturas tiverem o mesmo nome no mesmo segundo, o Specsify adiciona um sufixo sem sobrescrever a anterior.

## O que o arquivo organiza

- metadados e integridade do texto original.
- texto original completo.
- resumo processado.
- problema ou oportunidade.
- pessoas e valor percebidos.
- sinais de escopo, regras ou solução.
- dependências, falhas possíveis e direções futuras.
- pontos a revisar no futuro.
- rastreabilidade para backlog ou spec derivados.

Declarações, inferências e lacunas permanecem identificadas. Um campo sem base no texto aparece como não identificado, em vez de receber uma resposta inventada.

O texto será versionado no Git. Não inclua senhas, tokens, chaves privadas ou dados pessoais sensíveis. Se o agente detectar um segredo evidente, ele não grava a captura e orienta você a remover o conteúdo sensível antes de reenviar.

## Inbox, backlog e spec

captura sem perguntas → backlog refinável → spec normativa

- `specs/inbox/` preserva o input.
- `specs/backlog/` organiza algo escolhido para refinamento.
- `specs/specs/<NNNN>-<slug>/spec.md` governa comportamento e entrega.

Uma captura pode permanecer indefinidamente na Inbox. Quando quiser avançar, use `$specsfy-02-backlog` com o caminho do arquivo.

## Templates instalados

O CLI mantém em `.specsfy/templates/` os modelos usados para criar entradas, backlogs, specs e informações permanentes do projeto:

```
Inbox.md
Backlog.md
Spec.md
Tasks.md
Project.md
Stack.md
Rules.md
Database.md
```

Para personalizar um modelo, copie somente o arquivo desejado para `.specsfy/templates/custom/` e preserve o mesmo nome. A versão em `custom/` tem precedência sobre a cópia padrão. O instalador protege alterações locais nos templates gerenciados, mas `--force` pode substituí-los. O conteúdo de `custom/` nunca é gerenciado nem sobrescrito.

# Refinar ideias no backlog

---

O backlog recebe uma ideia que merece conversa, mas ainda não está pronta para virar especificação. Nele, você esclarece o problema, o público afetado e o efeito esperado sem definir arquitetura, tarefas ou código nessa etapa.

Para apenas guardar um texto sem responder perguntas, use a [Inbox](#). Depois que uma entrada for promovida, a `spec.md` passa a governar o comportamento, os gates, as tarefas e as evidências da entrega.

## Estrutura

```
specs/  
├─ inbox/  
│   └─ 2026-07-28-143205-ideia.md  
├─ backlog/  
│   └─ 0001-ideia.md  
└─ specs/  
    └─ 0001-feature/  
        ├─ spec.md  
        └─ research/
```

O item começa leve e amadurece até produto, desenvolvimento e testes compreenderem o comportamento esperado. Ele não contém tarefas ou gates e nunca autoriza implementação diretamente.

## Organização do backlog

```
Produto  
└─ Épico  
    └─ Funcionalidade  
        ├─ História ou requisito  
        ├─ Regra  
        ├─ Item técnico  
        └─ Melhoria
```

O épico representa um objetivo amplo. A funcionalidade delimita uma capacidade. Histórias e requisitos representam entregas menores e verificáveis. Regras, itens técnicos e melhorias também pertencem ao backlog quando tornam o comportamento ou sua operação possível.

Nem toda ideia precisa começar com a hierarquia completa. Use `A esclarecer` no lugar de inventar uma relação.

## Anatomia de um item

Logo abaixo do título, mantenha as metainformações em uma tabela, não em lista:

| METAINFORMAÇÃO | VALOR                       |
|----------------|-----------------------------|
| ID             | <code>BACKLOG-0001</code>   |
| Status         | <code>Captured</code>       |
| Produto        | <code>A esclarecer</code>   |
| Épico          | <code>A esclarecer</code>   |
| Funcionalidade | <code>A esclarecer</code>   |
| Tipo           | <code>A esclarecer</code>   |
| Prioridade     | <code>Não priorizado</code> |
| Criado em      | <code>data ISO</code>       |
| Spec promovida | <code>Nenhuma</code>        |

Conforme a conversa avança, o item pode registrar estas informações sem preencher campos que ainda não foram esclarecidos:

- título, tipo e prioridade.
- produto, épico e funcionalidade.
- contexto do problema.
- objetivo ou história do usuário.
- comportamento esperado.
- regras de negócio.
- condições de aceite.
- segurança, privacidade, desempenho, volume e auditoria.
- dependências.
- situações de erro e exceções.
- dentro e fora de escopo.

A profundidade é adaptativa. Uma alteração simples exige menos detalhes, enquanto autenticação, pagamentos, permissões, privacidade e processamento assíncrono exigem análise cuidadosa. O melhor item não é o mais longo, mas aquele que reduz a ambiguidade e permite verificar a entrega.

## Captura mínima e conversa

Ao receber a ideia, `$specsfy-02-backlog` reaproveita a descrição original e confirma:

- problema percebido.
- pessoa afetada ou beneficiada.
- resultado ou valor esperado.
- contexto suficiente para distinguir a ideia.

Se algo estiver ausente, vago, contraditório ou ambíguo, a skill pergunta uma lacuna relevante por vez e reavalia após cada resposta. Ela não usa questionário fixo, não repete o que já foi explicado e não persiste placeholders nesses campos. Se a pessoa não souber responder, explicita a lacuna sem inventar.

Esse é o mínimo de captura, não um refinamento profundo. Hierarquia, prioridade, regras detalhadas, aceite e solução técnica podem ser refinados depois.

## Duplicatas e referências

Antes de criar, a skill pesquisa termos da ideia em `specs/backlog/*.md`, `specs/specs/*/spec.md` e `docs/**/*.*md`. Ela separa possível duplicata de backlog relacionado, spec relacionada ou documentação útil.

Uma possível duplicata exige confirmar se o item existente será atualizado ou se há uma diferença real. Fontes úteis ficam em `Referências relacionadas`, com o caminho e o tipo de relação. Elas não substituem o que você declarou.

## Padrões recorrentes

| CAPACIDADE     | INFORMAÇÕES QUE O BACKLOG DEVE TORNAR OBJETIVAS       |
|----------------|-------------------------------------------------------|
| Autenticação   | cadastro, tentativas, sessão, mensagens e autorização |
| Notificações   | propriedade, leitura, ações em lote e isolamento      |
| Permissões     | perfil, operação, alvo, validação e auditoria         |
| PIX/pagamentos | estados, idempotência, webhook e dados sensíveis      |
| Exportação     | filtros, autorização, volume, fila e expiração        |

Use a representação que reduz ambiguidade. Fluxos numerados ajudam em integrações, matrizes ajudam em permissões e cenários demonstram resultados e erros.

Um requisito funcional descreve o que o sistema faz. Um requisito não funcional define condições mensuráveis de qualidade e operação. Troque “o sistema deve ser rápido” por um limite de latência, volume e ambiente verificáveis.

## Ordenar o backlog

Mantenha o backlog realmente ordenado para que a próxima oportunidade fique visível no arquivo para todos os responsáveis. Compare os itens pelos fatores abaixo:

1. valor para a pessoa e para o negócio.
2. exposição de segurança, privacidade e operação.
3. dependências e entregas desbloqueadas.
4. urgência.
5. esforço.
6. pontos ainda desconhecidos.

Prioridade é relativa. Classificar tudo como alta não cria uma ordem. Autenticação e autorização podem preceder melhorias visuais, assim como uma infraestrutura de notificações pode preceder os eventos que dependem dela.

A tabela abaixo mostra como uma ordem real diferencia o que deve ser retomado primeiro do que pode esperar:

| ORDEM | PRIORIDADE | TIPO     | ITEM                        | OBJETIVO                        |
|-------|------------|----------|-----------------------------|---------------------------------|
| 1     | Alta       | Épico    | Gestão de usuários          | Administrar acesso              |
| 2     | Alta       | História | Realizar login              | Acessar áreas privadas          |
| 3     | Alta       | Regra    | Controlar permissões        | Restringir operações por perfil |
| 4     | Média      | Técnico  | Notificações em fila        | Evitar lentidão nas requisições |
| 5     | Baixa      | Melhoria | Preferências de notificação | Escolher canais                 |

Cada linha aponta para um item próprio. Por exemplo, “realizar login” deve esclarecer cadastro ativo, comparação de e-mail, proteção da senha, tentativas inválidas, sessão, permissões, resultados de sucesso e falha e o que ficou fora do escopo. “Criar uma tela de login” não cobre esse comportamento.

## Fluxo

input → inbox → backlog → spec

1. Use `$specsfy-02-backlog` quando a entrada ainda for geral. A skill pesquisa material relacionado, conversa até a captura mínima ficar clara e produz `specs/backlog/<NNNN>-<slug>.md`. A mesma skill pode organizar, priorizar e refinar o item progressivamente.
2. A mesma skill faz uma pergunta relevante por vez e produz um brief quando houver decisões materiais abertas.
3. Use `$specsfy-03-specify` somente quando houver intenção explícita de promover o material. A fonte normativa é criada em `specs/specs/<NNNN>-<slug>/spec.md`.
4. Depois da promoção, mantenha o backlog como proveniência, marque-o `Promoted` e registre o caminho da spec.

Ao concluir, a skill informa qual etapa assumirá o trabalho e executa automaticamente o avanço ou o retorno. A conversa mostra o nome da skill, o motivo da troca e o resultado esperado. Você acompanha essa transição sem repetir comandos, e nenhuma troca autoriza implementação.

Backlog e spec possuem sequências independentes. `BACKLOG-0004` pode originar `SPEC-0002`, mas os números não precisam coincidir e não indicam prioridade.

## Estados do backlog

| ESTADO                  | SIGNIFICADO                                    |
|-------------------------|------------------------------------------------|
| Captured                | ideia preservada com contexto mínimo           |
| Refining                | conversa ou pesquisa leve em andamento         |
| Ready for specification | contexto suficiente para criar a especificação |
| Promoted                | spec derivada criada e referenciada            |

## Quando está refinado

O item está pronto para especificação quando a equipe consegue responder às perguntas abaixo sem consultar suposições fora do arquivo:

- Qual problema será resolvido e qual público será beneficiado?
- Qual evento inicia o comportamento e qual resultado será produzido?
- Qual papel pode executar a operação?
- Quais regras, erros e exceções precisam ser respeitados?

- Como verificar objetivamente o resultado?
- Há implicações de segurança, privacidade, desempenho ou volume?
- O que ficou fora da entrega?
- Quais dependências ou definições continuam pendentes?

O agente identifica lacunas e pergunta uma por vez. Definições que alteram segurança, escopo, arquitetura ou experiência não são inventadas silenciosamente. Esse diagnóstico prepara a spec, mas ainda não autoriza desenvolvimento.

## Limites

- Não implementar diretamente de um backlog.
- Não exigir arquitetura, Gherkin ou plano técnico durante a captura inicial.
- Não apagar a formulação original ao resumir.
- Não promover automaticamente uma ideia.
- Não tratar o backlog como fonte normativa depois da promoção.

## Próximo passo

Quando o item estiver claro o bastante para uma conversa aprofundada, use [specsfy-02-backlog](#). A promoção para `spec.md` só acontece depois de refinamento suficiente e de uma instrução explícita para criar a especificação.

# Skills base

---



As skills base dividem o método por responsabilidade. Você pode chamar uma delas pelo nome ou explicar o resultado esperado. O agente lê o estado da spec, seleciona a etapa responsável e anuncia cada transição necessária.

| ETAPA                                       | SKILL                                | RESULTADO PRINCIPAL                                     |
|---------------------------------------------|--------------------------------------|---------------------------------------------------------|
| capturar sem perguntas                      | <a href="#">specsfy-01-inbox</a>     | arquivo em <code>specs/inbox/</code>                    |
| refinar uma entrada e aprofundar definições | <a href="#">specsfy-02-backlog</a>   | item em <code>specs/backlog/</code> e brief na conversa |
| criar a fonte única                         | <a href="#">specsfy-03-specify</a>   | <code>spec.md</code>                                    |
| revisar definição                           | <a href="#">specsfy-04-validate</a>  | Definition Gate confiável                               |
| decompor o plano                            | <a href="#">specsfy-05-tasks</a>     | tarefas dentro da spec                                  |
| preparar e executar testes                  | <a href="#">specsfy-06-tdd-bdd</a>   | RED/GREEN rastreável                                    |
| produzir a mudança                          | <a href="#">specsfy-07-implement</a> | código, testes e evidência                              |
| incorporar mudança posterior                | <a href="#">specsfy-update-spec</a>  | spec atualizada e gates reabertos                       |
| consultar o estado                          | <a href="#">specsfy-progress</a>     | relatório somente leitura                               |

## Encontre a skill pelo estado do trabalho

Uma anotação que precisa ser preservada vai para `specsfy-01-inbox`. Quando você quiser comparar essa ideia com itens existentes e esclarecer o mínimo necessário, `specsfy-02-backlog` cria ou atualiza o arquivo numerado e aprofunda as definições que mudam o comportamento e entrega um brief na conversa.

Com a intenção de criar uma entrega confirmada, `specsfy-03-specify` monta a `spec.md` e `specsfy-04-validate` comprova o Ato I. A skill `specsfy-05-tasks` organiza o plano, chama `specsfy-06-tdd-bdd` para materializar os testes e só aprova o Plan Gate depois de um RED válido. `specsfy-07-implement` executa as tarefas com evidência e documentação atualizada.

Uma necessidade surgida depois da definição retorna à mesma spec por `specsfy-update-spec`. Para apenas consultar gates, tarefas e o próximo trabalho sem alterar arquivos, use `specsfy-progress`.

Volte ao [guia completo](#) ou leia [como a metodologia funciona](#).

# Capturar uma entrada com `specsfy-01-inbox`

---

Esta skill funciona como uma caixa de entrada: recebe seu texto, guarda o original e organiza uma primeira leitura sem interromper você com perguntas.

## Quando usar

Use quando quiser anotar uma entrada, oportunidade, necessidade ou pensamento para retomar depois. O arquivo fica em `specs/inbox/`.

Não use para refinar prioridade, decidir requisitos ou iniciar implementação. Esses trabalhos pertencem às etapas seguintes.

## Como descrever a tarefa

Use `$specsfy-01-inbox` para capturar:  
seria útil avisar clientes quando uma entrega atrasar, talvez por e-mail.

Se preferir uma instrução curta, escreva “guarde na Inbox” e inclua o texto original na mesma mensagem. A skill organiza a captura sem exigir um formato prévio.

## Exemplo passo a passo

1. Você envia o texto livre.
2. O agente preserva o texto original.
3. Sem fazer perguntas, ele separa o resumo, o problema ou a oportunidade, as pessoas afetadas, o resultado esperado, as dependências e os pontos que ainda precisam de revisão.
4. Ele cria um nome com data, hora e slug.
5. Ele informa o caminho criado e encerra a captura.

## O que esperar

`specs/inbox/2026-07-28-143205-avisar-clientes-sobre-atrasos.md`

O arquivo diferencia o que você declarou, o que foi inferido e o que ainda precisa ser revisto. Nenhum backlog, spec, tarefa ou código é criado.

Os metadados incluem horário da captura, origem, slug, status, hash do texto original e links futuros para backlog ou spec.

## Erros comuns

- esperar que a captura faça perguntas ou complete definições ausentes.
- tratar a análise inicial como requisito aprovado.
- implementar diretamente a partir de `specs/inbox/`.
- editar o texto original em vez de registrar uma evolução no backlog.
- procurar o template dentro da skill: o padrão vive em `.specsfy/templates/Inbox.md`, e uma personalização homônima em `.specsfy/templates/custom/` tem precedência.

## Próximo passo

Deixe a entrada guardada ou use [specsfy-02-backlog](#) quando quiser refiná-la.

# Refinar uma entrada com `specsfy-02-backlog`

---

Esta skill transforma uma entrada da Inbox em backlog refinável. Ela procura itens parecidos, registra o contexto inicial e, quando necessário, conduz uma pergunta prioritária por vez até produzir um brief pronto para especificar.

## Quando usar

Use quando quiser organizar uma captura em `specs/inbox/`, avaliar uma oportunidade ou fechar lacunas sobre público, finalidade, regras, limites, privacidade, falhas e resultado esperado. Para apenas salvar sem perguntas, use `specsfy-01-inbox`.

Não use para planejar tarefas, escrever testes ou iniciar código, pois um item de backlog ainda não passou pelos gates que autorizam essas etapas.

## Como descrever a tarefa

Descreva a entrada e o destino esperado na mesma mensagem. A skill usará esse texto para criar um item reconhecível em `specs/backlog/`, sem tratá-lo como autorização para implementar. Por exemplo:

```
Use $specsfy-02-backlog para refinar esta entrada:
permitir que a pessoa escolha o idioma da interface.
```

Quando houver itens parecidos em `specs/backlog/`, inclua a situação que diferencia esta entrada das demais. Isso ajuda a skill a atualizar o item correto e a manter visíveis as definições que continuam abertas:

```
Anote no backlog: clientes internacionais não entendem os e-mails atuais.
Ainda não decidimos quais idiomas serão oferecidos.
```

## Exemplo passo a passo

1. Você apresenta a entrada ou aponta o arquivo da Inbox.
2. O agente também pode ler a origem em `specs/inbox/`.
3. Ele procura itens semelhantes em `specs/backlog/` e nas specs.
4. Ele pergunta uma lacuna material por vez.
5. Você responde: “o problema afeta e-mails e a interface”.

6. A skill cria o item com `.specsfy/templates/custom/Backlog.md` quando presente ou com `.specsfy/templates/Backlog.md` :

```
specs/backlog/0003-idioma-da-interface.md
```

O item registra problema, público, resultado esperado e dúvidas abertas. Ele continua sendo backlog e não autoriza implementação.

## Como o ciclo termina

O refinamento funciona sem limite máximo de perguntas. A cada resposta, o agente reconsidera o pedido original, o contexto acumulado e a nova resposta. Ele continua enquanto existir uma lacuna aplicável, sem seguir uma lista fixa.

A partir da 11ª pergunta, cada rodada também oferece `avançar` . Essa opção permite encerrar o ciclo atual mesmo com decisões abertas. Nesse caso, o brief lista as lacunas, a spec permanece `Status: Draft` e o `Definition Gate: Pending` ; `avançar` não equivale a aprovar a definição.

## O que esperar

- perguntas adaptadas ao caso, uma lacuna importante por vez.
- preservação das suas palavras.
- indicação de duplicatas ou relações.
- distinção entre fato, hipótese e escolha confirmada.
- um brief pronto para especificar.
- um caminho claro para retomar depois.
- nenhuma `spec.md` criada automaticamente sem promoção.

## Erros comuns

- transformar o backlog em uma especificação completa.
- inventar solução técnica quando o problema ainda está aberto.
- criar um segundo item sem procurar duplicatas.
- tratar o item como aprovação para implementar.
- responder categorias como um formulário fixo.
- esconder uma dúvida para encurtar a conversa.

## Próximo passo

Quando o backlog estiver claro, use [specsfy-03-specify](#). Se ainda houver uma decisão material aberta, continue o ciclo nesta mesma skill.

# Criar a especificação com `specsfy-03-specify`

---

Esta skill cria a fonte única da entrega no arquivo `spec.md`. Ao abrir esse arquivo, você encontra a descoberta usada como base para os requisitos, os comportamentos que orientam o plano e a ligação entre tarefas, testes e evidências. Essa concentração evita que arquivos paralelos apresentem versões diferentes da mesma entrega.

## Quando usar

Use para promover um backlog refinado ou criar uma spec nova ainda em estado Draft. Para mudar uma spec que já foi definida, use `update-spec`.

## Como descrever a tarefa

Quando a ideia já tiver sido refinada no backlog, informe o caminho do item:

```
Use $specsfy-03-specify para promover
specs/backlog/0003-idiotoma-da-interface.md.
```

Quando o refinamento do backlog tiver produzido um brief na conversa, peça a criação com base nas definições registradas:

```
Use $specsfy-03-specify para criar uma especificação para recuperação de senha
com base nas definições desta conversa.
```

## Exemplo passo a passo

1. A skill confirma a raiz do projeto e o próximo número disponível.
2. Ela lê as instruções, o brief e as evidências necessárias.
3. Cria o pacote:

```
specs/specs/0004-recuperar-senha/
├─ spec.md
├─ research/          # somente quando houver pesquisa externa
```

Em seguida, ela preenche o Ato I com requisitos e cenários, mantém as escolhas ainda abertas claramente marcadas e executa os validadores. Um resultado incompleto permanece pendente em vez de receber uma aprovação sem evidência. Quando falta uma decisão material, a skill entrega a conversa ao ciclo de refinamento do backlog e retoma

depois. Se você escolher `avançar`, a spec pode registrar o brief parcial, mas permanece Draft com o Definition Gate pendente. A mesma o refinamento não é reaberto imediatamente nessa retomada.

Os arquivos em `research/` apoiam as escolhas registradas, mas nunca substituem o texto normativo de `spec.md`. Quando houver divergência, a entrega e seus validadores devem seguir a spec.

## O que esperar

- formato `Specsfy/2.0`.
- IDs rastreáveis para histórias, requisitos e condições de aceite.
- cenários que cobrem sucesso, variação e falha.
- uma única fonte normativa.
- status Draft ou Defined conforme a evidência real.

## Erros comuns

- criar `plan.md`, `tasks.md` ou `research.md`.
- copiar uma fonte externa como requisito sem confirmação.
- aprovar gates com campos incompletos.
- misturar várias entregas grandes na mesma spec.

## Próximo passo

Use [specsfy-04-validate](#) para provar que a definição está pronta. Se uma escolha importante faltar, a transição volta para [specsfy-02-backlog](#).

# Validar a definição com `specsfy-04-validate`

---

Esta skill revisa a spec como código em linguagem natural. Primeiro verifica o formato. Depois avalia clareza, completude, consistência e possibilidade de teste.

## Quando usar

Use para comprovar o Ato I ou quando uma mudança exigir nova validação da definição. Também é útil para auditar uma spec sem alterar sua intenção.

## Como descrever a tarefa

```
Use $specsfy-04-validate em  
specs/specs/0004-recuperar-senha/spec.md.
```

## Exemplo passo a passo

1. A skill confirma o caminho e o formato `Specsfy/2.0`.
2. Verifica se os requisitos possuem exemplos suficientes.
3. Encontra uma lacuna: “a validade do link não foi decidida”.
4. Retorna para o refinamento do backlog e registra a validade de 30 minutos.
5. Executa novamente os validadores.
6. Atualiza a seção de gates:

```
Definition Gate: Passed  
Status: Defined
```

Uma falha de parser, fixture ou ambiente não serve como prova de requisito ausente.

## O que esperar

- problemas apontados com localização e motivo.
- nenhuma aprovação baseada em suposição.
- verificação das evidências externas citadas.
- retorno automático à etapa que pode resolver a lacuna.
- gate aprovado somente após nova validação.

## Erros comuns

- marcar READY sem resolver uma ambiguidade material.
- confundir estrutura válida com conteúdo suficiente.
- criar um relatório paralelo em vez de atualizar a seção correta.
- compensar um Ato I incompleto em uma etapa posterior.

## Próximo passo

Com o Definition Gate aprovado, use [specsfy-05-tasks](#) . Se a validação encontrar uma escolha aberta, use [specsfy-02-backlog](#) .

# Planejar tarefas com `specsfy-05-tasks`

Esta skill transforma uma definição aprovada em tarefas pequenas, ordenadas e verificáveis. As tarefas ficam na seção 14 da própria `spec.md`.

## Quando usar

Use depois do Definition Gate ou quando uma alteração exigir replanejamento. Não use para escrever código nem para marcar uma tarefa como concluída.

## Como descrever a tarefa

Use `$specsfy-05-tasks` em `specs/specs/0004-recuperar-senha/spec.md`.

Se a spec contiver mais de uma entrega observável, indique a fatia vertical que deve ser planejada primeiro:

Prepare as tarefas da primeira fatia vertical da spec 0004.

## Exemplo passo a passo

1. A skill lê a spec e o código existente.
2. Identifica a menor entrega observável: solicitar o link.
3. Liga cada tarefa aos requisitos e às condições de aceite correspondentes.
4. Faz cada tarefa de produção depender de um teste com RED registrado.
5. Registra:

```
T001 [ ] Criar caso TDD para solicitação válida – cobre AC-001
T002 [ ] Criar caso TDD para e-mail desconhecido – cobre AC-002
T003 [ ] Implementar solicitação sem revelar existência do cadastro
```

Depois de registrar as tarefas, a skill chama `specsfy-06-tdd-bdd` para materializar os testes. O Plan Gate só pode ser aprovado quando todos os predecessores exigidos possuem RED válido.

## O que esperar

- tarefas pequenas e com resultado verificável.

- ordem explícita de dependência.
- testes com RED como predecessores do código.
- caminhos e comandos reais do projeto.
- tarefas mantidas dentro da fonte única.

## Erros comuns

- criar `tasks.md`.
- escrever tarefas vagas como “fazer backend”.
- colocar várias mudanças independentes em uma tarefa.
- planejar sem inspecionar a stack real.
- marcar uma tarefa pronta sem evidência.

## Próximo passo

Use [specsfy-06-tdd-bdd](#) em modo `prepare` para materializar o próximo teste e observar RED.

# Preparar testes com `specsify-06-tdd-bdd`

---

Esta skill usa os cenários da spec para criar testes executáveis. Ela mantém a rastreabilidade entre o comportamento descrito e a prova no código.

## Quando usar

Use para preparar o RED que autoriza a implementação, executar um ciclo RED-GREEN-REFACTOR ou verificar testes e rastreabilidade.

## Como descrever a tarefa

Para preparar o próximo teste focal e produzir a evidência de RED, use o modo `prepare` :

```
Use $specsify-06-tdd-bdd em modo prepare para
specs/specs/0004-recuperar-senha/spec.md.
```

Para conferir os testes e a rastreabilidade de uma entrega existente, use o modo `verify` :

```
Use $specsify-06-tdd-bdd em modo verify na spec 0004.
```

## Exemplo passo a passo

1. A skill seleciona a próxima condição de aceite.
2. Encontra o runner real do projeto.
3. Cria um teste com marcador de rastreabilidade.
4. Executa somente o teste focal.
5. Confirma:

```
RED válido: o teste falhou porque a recuperação ainda não foi implementada.
Caso: TDD-AC-001
```

Depois da implementação, a skill repete o teste focal e executa a regressão para confirmar que o comportamento ficou GREEN sem quebrar o restante.

O bloco Gherkin da spec é uma referência legível. A prova automatizada fica na suíte normal do projeto, não em um arquivo `.feature` nem em uma segunda suíte.

## O que esperar

- caso de teste ligado a uma condição de aceite da spec.
- comando e resultado registrados.
- distinção entre falha esperada e problema de ambiente.
- cobertura de sucesso, regra e limite.
- regressão depois do GREEN.

## Erros comuns

- chamar erro de configuração de RED.
- escrever produção no modo `prepare`.
- criar testes que não correspondem às condições de aceite.
- considerar o Gherkin sozinho como teste executado.
- aprovar o plano sem prova focal.

## Próximo passo

Com RED válido e tarefa pronta, use [specsfy-07-implement](#). Para apenas reorganizar as tarefas, volte a [specsfy-05-tasks](#).

# Entregar código com `specsfy-07-implement`

Esta skill executa as tarefas aprovadas da spec em ordem. Ela altera produção somente quando existe um plano válido e um teste focal em RED.

## Quando usar

Use para implementar a próxima tarefa pronta, continuar uma entrega ou concluir uma feature planejada. Não use para pular definição, planejamento ou testes.

## Como descrever a tarefa

Use `$specsfy-07-implement` para executar a próxima tarefa pronta de `specs/specs/0004-recuperar-senha/spec.md`.

Quando houver mais de uma tarefa pronta, indique o ID da tarefa que deve ser executada:

Implemente T003 da spec 0004 e valide a regressão.

## Exemplo passo a passo

1. A skill confirma Definition Gate e Plan Gate aprovados.
2. Verifica a tarefa predecessora e o RED atual.
3. Faz a menor mudança de produção.
4. Executa o teste focal até obter GREEN.
5. Refatora sem alterar o comportamento.
6. Executa a regressão e atualiza evidências:

T003 [x] Implementar solicitação sem revelar existência do cadastro  
Teste focal: passou  
Regressão: passou

Depois de cada tarefa de código, a skill chama o documentador do projeto consumidor. A execução só continua quando `docs/` estiver atualizado.

## O que esperar

- uma tarefa por vez.
- mudanças limitadas ao escopo aprovado.

- testes focais e regressão.
- documentação aplicável atualizada.
- status e checkboxes comprovados por evidência.

## Erros comuns

- implementar com gate pendente.
- aceitar um RED causado por dependência ausente.
- ampliar o escopo sem atualizar a spec.
- marcar conclusão sem regressão.
- deixar `docs/` , `PROJECT.md` ou os arquivos `.specsfy/` incompatíveis com o código alterado.

## Próximo passo

Continue com a próxima tarefa ou consulte [specsfy-progress](#) . Se surgir uma necessidade nova, use [specsfy-update-spec](#) .

# Incorporar mudanças com `specsify-update-spec`

---

Esta skill atualiza uma spec que já foi definida, planejada, iniciada ou concluída. A nova necessidade entra na mesma fonte, e somente os atos cujas provas perderam validade são reabertos.

## Quando usar

Use quando uma entrega existente precisar incorporar algo esquecido, adicionar ou remover comportamento, corrigir uma definição ou mudar uma regra já registrada.

Para criar a primeira versão da spec, use `specify`. A `update-spec` depende de uma fonte existente para comparar a nova instrução com requisitos, testes, tarefas e gates já registrados.

## Como descrever a tarefa

Use `$specsify-update-spec` para adicionar expiração de 30 minutos à `specs/specs/0004-recuperar-senha/spec.md`.

Para remover um comportamento, identifique a exigência e o arquivo `spec.md` que deve ser revisto. Assim, a skill consegue localizar testes e tarefas que ficariam incompatíveis com a remoção:

Remova a exigência de SMS da spec 0004 e ajuste o trabalho afetado.

## Exemplo passo a passo

1. A skill preserva literalmente a nova instrução.
2. Lê requisitos, testes, tarefas, gates e evidências atuais.
3. Classifica a mudança:

Mudança de definição: altera comportamento e condição de aceite.  
Atos reabertos: I, II e III.

Com o impacto identificado, a skill atualiza a mesma `spec.md` e invalida somente os gates que perderam validade. Depois, retoma refinamento, validação, tarefas, testes e implementação na ordem necessária. A nova evidência é registrada sem apagar o histórico relevante.

Se a mudança abrir decisões materiais, o refinamento do backlog pergunta uma lacuna por vez e reavalia o contexto após cada resposta. O ciclo não possui limite. A partir da 11ª pergunta, `avancar` encerra o refinamento do backlog atual, preserva as lacunas e mantém o Ato I reaberto, sem iniciar outra vez o mesmo ciclo nessa retomada.

## O que esperar

- instrução original preservada.
- impacto explicado na retomada.
- nenhuma spec duplicada.
- trabalho já válido mantido.
- transições automáticas até o estado coerente.

## Erros comuns

- editar apenas o código e deixar a spec antiga.
- reabrir todos os gates sem analisar impacto.
- manter teste ou tarefa que contradiz a nova instrução.
- esconder a mudança em notas soltas.
- criar uma segunda especificação para a mesma entrega.

## Próximo passo

A própria skill encaminha para a etapa reaberta. Depois da atualização, use [specsfy-progress](#) para revisar o estado geral.

# Consultar o estado com `specsfy-progress`

Esta skill lê todas as specs e apresenta uma visão geral de gates, tarefas, checklists, falhas que impedem avanço e próximo trabalho. Ela não altera nenhum estado.

## Quando usar

Use para saber quanto falta, qual falha impede uma entrega, qual tarefa está pronta ou quais specs foram concluídas.

## Como descrever a tarefa

Para receber uma síntese na conversa, descreva o projeto que deseja consultar:

```
Use $specsfy-progress para mostrar o progresso do projeto.
```

No terminal, use o CLI para obter a mesma leitura das fontes canônicas:

```
specsfy progress --project .
```

## Exemplo passo a passo

1. A skill lê `specs/specs/*/spec.md`.
2. Calcula o estado a partir de gates e checkboxes.
3. Não consulta um relatório paralelo.
4. Apresenta:

```
Specs: 2
Complete: 1
Implementing: 1
Tarefas: 7 de 10 concluídas
Próximo trabalho: T004 da spec 0004-recuperar-senha
Pendências impeditivas: nenhuma
```

Se os metadados estiverem incoerentes, o relatório mostra a pendência em vez de inventar um percentual confiável.

## O que esperar

- leitura somente das fontes canônicas.

- totais de specs e tarefas.
- gates pendentes.
- falhas impeditivas e próximo trabalho.
- saída humana ou JSON pelo CLI.

## Erros comuns

- alterar checkboxes durante a consulta.
- manter um segundo arquivo de progresso.
- calcular conclusão só pelo número de arquivos.
- esconder spec inválida do relatório.
- confundir progresso com autorização para implementar.

## Próximo passo

Abra a página da skill indicada pelo próximo trabalho. Para acompanhar continuamente no terminal, use:

```
specsfy progress --project . --watch
```

# CLI e TUI do Specsify

---

O executável `specsify` instala e atualiza as skills, mostra o progresso, executa os testes detectados e abre um dashboard no terminal. Instale primeiro o CLI e o framework seguindo o [guia de instalação](#). Este guia assume que `specsify --version` já responde no terminal e que o bootstrap foi executado no projeto consumidor. Para conduzir a primeira fatia depois do bootstrap, siga o [guia do primeiro projeto](#). Para seleção técnica, automação e reabertura de gates, consulte o [uso avançado](#).

Os templates de ideia, backlog, spec, tarefas e informações permanentes ficam em `.specsify/templates/`. Customizações com o mesmo nome ficam em `.specsify/templates/custom/` e têm precedência sobre os arquivos padrão. Ao criar uma especificação, `specsify-03-specify` renderiza o template resolvido em `specs/specs/<NNNN>-<slug>/spec.md`. O exemplo demonstra os três atos e as 18 seções para agentes, testes e diagnóstico do CLI, mas não governa uma feature. O cabeçalho renderizado é uma tabela Markdown de duas colunas, `Campo` e `Valor`, com um metadado por linha.

## Instalar e gerenciar skills

Para preparar um projeto novo, o comando `install` pode publicar as bases e todos os especialistas detectados em uma única execução:

```
specsify install --project . --detected
```

Quando você já souber quais stacks precisam de orientação especializada, repita `--specialist` para instalar somente o conjunto indicado:

```
specsify install --project . \  
  --specialist specsify-specialist-laravel \  
  --specialist specsify-specialist-postgres
```

Depois da instalação inicial, os subcomandos de `skills` permitem listar o catálogo, detectar recomendações, adicionar uma skill ou atualizar as versões gerenciadas:

```
specsify skills list  
specsify skills detect --project .  
specsify skills add specsify-specialist-laravel --project .  
specsify skills update --project .
```

## Atualização automática

Ao abrir `specsify` ou `specsify tui` em um terminal interativo, o CLI verifica as tags semânticas estáveis de `cli/`. O cache e as configurações globais ficam em `~/.specsify/cli.json`, com permissão restrita ao usuário e intervalo padrão de 24 horas entre consultas.

Quando uma tag aponta para uma versão superior, o CLI apresenta a versão atual e pergunta se deve atualizar. Recusar abre o dashboard normalmente. Aceitar executa `npm install --global @promovaweb/specsify@latest` e encerra o CLI. A versão nova entra em uso na próxima abertura.

O arquivo global separa `settings`, incluindo habilitação e intervalo da consulta, de `cache`, que registra horário, tag, versão, commit, ETag e erro recente. Chaves desconhecidas são preservadas. A aplicação continua abrindo quando a rede está indisponível, a resposta é inválida ou a escrita falha, e o aviso fica para a próxima consulta.

Como o monorepo é privado, catálogo e tags são consultados com `GH_TOKEN`, `GITHUB_TOKEN` ou, na ausência dessas variáveis, com a sessão de `gh auth token`. O token não é copiado para `~/.specsify/cli.json`.

Em uma instalação global gerenciada pelo npm, o comando abaixo atualiza o pacote. Abra o CLI novamente para conferir a nova versão:

```
npm update --global @promovaweb/specsify
```

Se você instalou o executável Node com `curl -fL get.specsify.dev`, repita o download descrito no [guia de instalação](#). Nesse caso, a oferta automática exige o npm disponível; uma falha preserva a versão atual e abre a TUI normalmente.

## Dashboard e progresso

Execute sem argumentos dentro do projeto consumidor para abrir a TUI:

```
specsify
```

`specsify tui --project PATH` abre explicitamente outro projeto. O dashboard é organizado em seis abas:

- **Home**, com as estatísticas consolidadas.
- **Backlogs**, com lista navegável na coluna esquerda e preview Markdown formatado na coluna direita.
- **Specs**, com a tabela e o progresso de cada especificação.
- **Testes**, com execução do runner detectado, resumo da última execução e saída detalhada em subabas separadas.

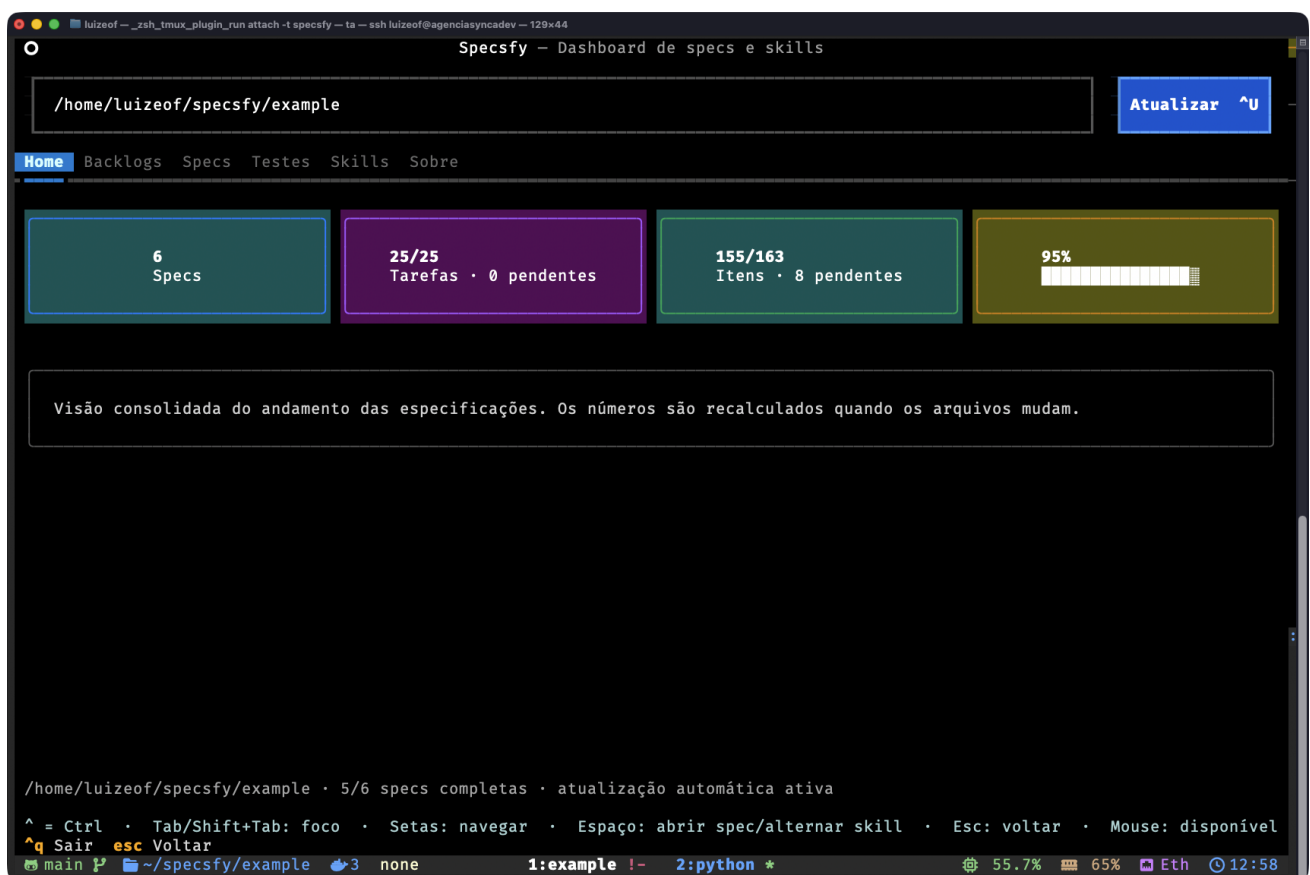
- **Skills**, com catálogo tabular, painel de detalhes e uma prévia explícita das instalações e remoções pendentes.
- **Sobre**, com a versão e a finalidade do CLI.

No dashboard, os cards e as tabelas combinam estes dados para mostrar tanto o estado global quanto a situação de cada spec:

- quantidade total e concluída de specs.
- tarefas T... concluídas, pendentes e totais.
- todos os itens de checklist concluídos, pendentes e totais.
- gates aprovados por spec.
- porcentagem e barra de progresso global.
- porcentagem e barra de progresso de cada spec.

## Percurso visual

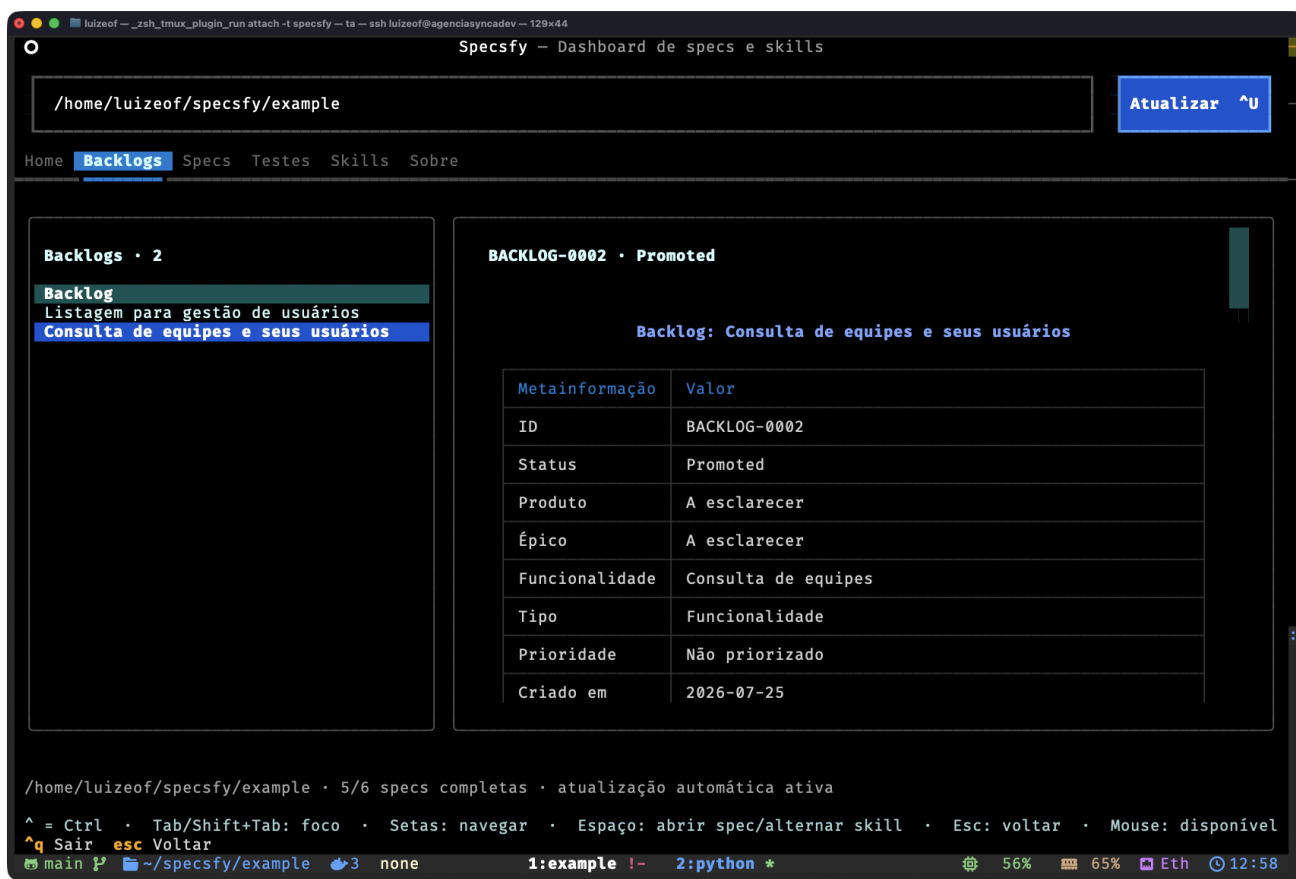
### Home: visão consolidada



Dashboard Home

A Home reúne o total de specs, o avanço das tarefas e checklists e a porcentagem global. O diretório selecionado aparece no topo e o rodapé confirma quantas specs estão completas e se a atualização automática está ativa.

## Backlogs: lista e leitura lado a lado



Specsfy — Dashboard de specs e skills

/home/luizeof/specsfy/example Atualizar ^U

Home **Backlogs** Specs Testes Skills Sobre

**Backlogs · 2**

- Backlog
- Listagem para gestão de usuários
- Consulta de equipes e seus usuários**

**BACKLOG-0002 · Promoted**

Backlog: Consulta de equipes e seus usuários

| Metainformação | Valor               |
|----------------|---------------------|
| ID             | BACKLOG-0002        |
| Status         | Promoted            |
| Produto        | A esclarecer        |
| Épico          | A esclarecer        |
| Funcionalidade | Consulta de equipes |
| Tipo           | Funcionalidade      |
| Prioridade     | Não priorizado      |
| Criado em      | 2026-07-25          |

/home/luizeof/specsfy/example · 5/6 specs completas · atualização automática ativa

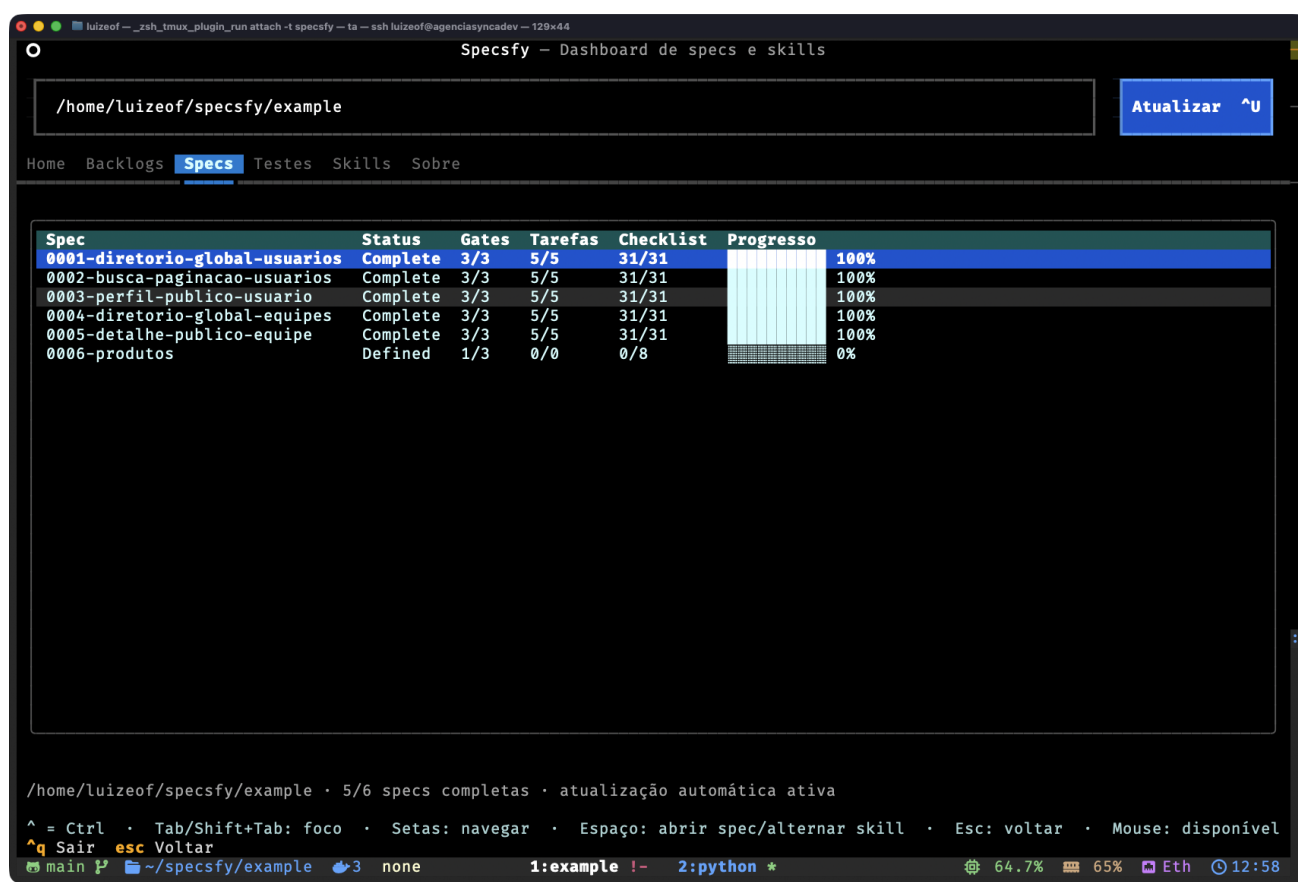
^ = Ctrl · Tab/Shift+Tab: foco · Setas: navegar · Espaço: abrir spec/alternar skill · Esc: voltar · Mouse: disponível  
^q Sair **esc** Voltar

main ~/specsfy/example 3 none 1:example !- 2:python \* 56% 65% Eth 12:58

Backlogs

A aba Backlogs mantém a seleção na coluna esquerda e renderiza o Markdown do item na direita. Assim é possível conferir metainformação, status e conteúdo sem abandonar o dashboard.

## Specs: gates, tarefas e progresso



Specsfy – Dashboard de specs e skills

/home/luizeof/specsfy/example Atualizar ^U

Home Backlogs **Specs** Testes Skills Sobre

| Spec                           | Status   | Gates | Tarefas | Checklist | Progresso |
|--------------------------------|----------|-------|---------|-----------|-----------|
| 0001-diretorio-global-usuarios | Complete | 3/3   | 5/5     | 31/31     | 100%      |
| 0002-busca-paginacao-usuarios  | Complete | 3/3   | 5/5     | 31/31     | 100%      |
| 0003-perfil-publico-usuario    | Complete | 3/3   | 5/5     | 31/31     | 100%      |
| 0004-diretorio-global-equipes  | Complete | 3/3   | 5/5     | 31/31     | 100%      |
| 0005-detalle-publico-equipe    | Complete | 3/3   | 5/5     | 31/31     | 100%      |
| 0006-produtos                  | Defined  | 1/3   | 0/0     | 0/8       | 0%        |

/home/luizeof/specsfy/example · 5/6 specs completas · atualização automática ativa

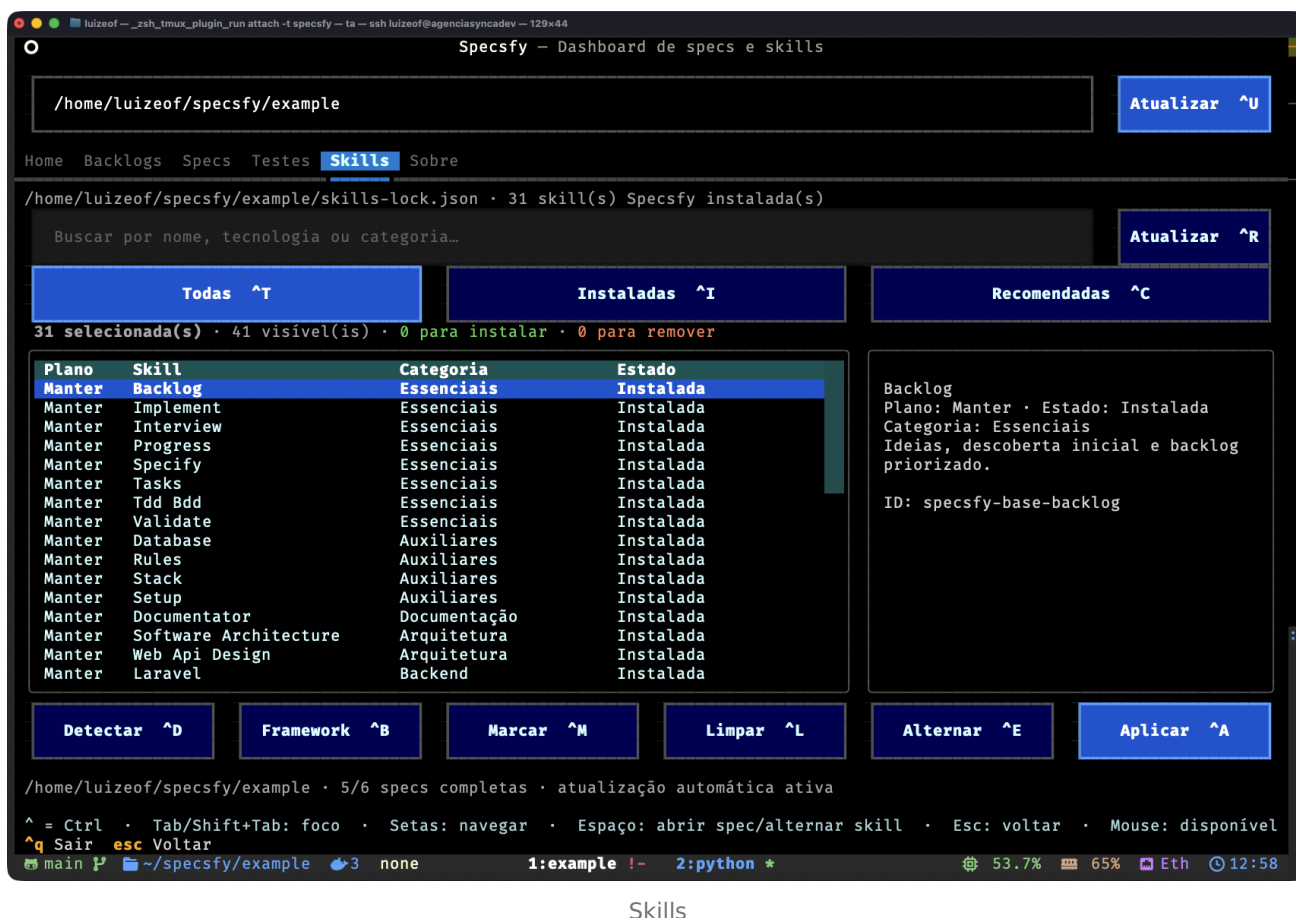
^ = Ctrl · Tab/Shift+Tab: foco · Setas: navegar · Espaço: abrir spec/alternar skill · Esc: voltar · Mouse: disponível  
^q Sair esc Voltar

main ~/specsfy/example 3 none 1:example !- 2:python \* 64.7% 65% Eth 12:58

Specs

A aba Specs compara status, gates, tarefas, checklists e porcentagem por especificação. A linha destacada pode ser aberta com **Espaço** para consultar a spec completa em um modal Markdown.

## Skills: planejar antes de aplicar



Skills

A aba Skills combina busca, filtros, plano, categoria e estado com o painel de detalhes da seleção. Os totais acima da tabela mostram instalações e remoções planejadas. Nada muda no projeto até a ação **Aplicar**.

## Testes do projeto

Em um projeto Laravel com Pest, `specsify test` detecta o runner e transmite a saída do teste no mesmo terminal:

```
specsify test --project .
```

O CLI detecta `artisan` e `pestphp/pest`, chama `php artisan test` diretamente na raiz selecionada, transmite a saída e preserva o exit code do runner. Ele não aceita uma string de shell arbitrária.

Na aba **Testes**, Executar testes `^X` inicia a mesma execução. A subaba **Resumo** mostra resultado, runner, comando, projeto, duração, exit code e os totais emitidos pelo Pest. A subaba **Testes** mantém a saída completa e rolável. Quando o projeto fornece um relatório Pest estruturado, cada falha é apresentada com nome do teste, arquivo, linha e mensagem.

O progresso usa todos os checkboxes Markdown de `specs/specs/*/spec.md`. Tarefas com ID `T...` também ganham estatística própria. Quando uma spec não possui checkboxes, os três gates são usados como projeção de fallback.

A TUI calcula fingerprints dos backlogs, das specs e do `skills-lock.json`. Ela atualiza automaticamente as listas, previews, cards e seleção de skills quando esses arquivos mudam. O intervalo padrão é 0,75 segundo e pode ser configurado por projeto:

```
specsfy config show --project .  
specsfy config set --project . --watch-interval 0.5
```

O rodapé apresenta os atalhos disponíveis na aba atual. Use essas combinações para trocar de tela ou aplicar uma ação sem retirar o foco do terminal:

- `Ctrl+Q` : sair.
- `Ctrl+U` : atualizar.
- `Ctrl+D` : detectar recomendações.
- `Ctrl+B` : selecionar todas as skills do framework.
- `Ctrl+E` : alternar o plano da skill destacada.
- `Ctrl+V` : marcar os resultados visíveis.
- `Ctrl+L` : limpar os resultados visíveis.
- `Ctrl+A` : aplicar a seleção.
- `Ctrl+R` : atualizar todas as skills Specsfy instaladas.
- `Ctrl+T`, `Ctrl+N`, `Ctrl+C` : abrir os filtros Todas, Instaladas e Recomendadas.
- `Ctrl+H`, `Ctrl+G`, `Ctrl+S`, `Ctrl+K`, `Ctrl+O` : abrir Home, Backlogs, Specs, Skills e Sobre.
- `Ctrl+J` : abrir Testes.
- `Ctrl+X` : executar os testes do projeto selecionado.

Os atalhos são globais e aparecem nos próprios rótulos dos botões. A interface inteira também aceita:

- `Tab` e `Shift+Tab` para percorrer controles.
- setas para navegar listas e tabelas.
- `Enter` ou `Espaço` para alternar o plano da skill destacada.
- `Esc` para fechar o modal da spec, limpar a busca ou voltar à Home.
- mouse para abas, listas, preview, filtros e botões.

O campo de projeto entra em edição com `Enter` ou com um clique. A confirmação recarrega backlogs, specs e skills a partir do caminho informado.

Foco, cursor, seleção e ações primárias usam contraste reforçado para manter legibilidade em terminais escuros.

Na aba Skills, cada linha separa Plano, Skill, Categoria e Estado. O plano usa os valores Instalar, Manter, Remover e Ignorar. Ele expressa o que acontecerá sem executar a mudança imediatamente. O painel lateral mostra o identificador completo, a descrição, a recomendação e o plano da linha destacada. A alteração só ocorre ao acionar Aplicar.

A configuração vive em <projeto>/specsfy/config.json. Valores desconhecidos adicionados pelo usuário são preservados quando o CLI atualiza uma opção.

Para automação, specsfy progress pode emitir texto, JSON ou uma sequência de snapshots. Assim, outro processo recebe uma nova leitura somente quando o conteúdo das specs muda:

```
specsfy progress --project .
specsfy progress --project . --json
specsfy progress --project . --watch
specsfy progress --project . --watch --interval 0.5 --json
```

O JSON contém um objeto summary e a coleção specs. Com --watch, um novo snapshot é emitido somente quando o conteúdo das specs muda, o que evita leituras repetidas em uma integração. A instalação das bases e dos especialistas continua disponível na TUI e nos comandos de skills.

O leitor mantém compatibilidade com specs/<NNNN>-<slug>/spec.md para projetos existentes, mas toda spec nova usa specs/specs/. Itens em specs/backlog/ não entram na porcentagem de entrega.

## Justificativa de tamanho

Comandos, dashboard e atalhos operam os mesmos arquivos do projeto. Reuni-los nesta página permite comparar a ação no terminal com o resultado mostrado na TUI, sem duplicar explicações sobre progresso e proteção de alterações locais.

## Segurança e reversibilidade

- O destino padrão é <projeto>/agents/skills.
- As regras centrais ficam em <projeto>/specsfy/Spec.md.
- Template e exemplo ficam em <projeto>/specsfy/templates/Spec.md e <projeto>/specsfy/examples/Spec.md.
- Templates personalizados ficam em <projeto>/specsfy/templates/custom/, fora do lock e protegidos até mesmo de operações com --force.
- AGENTS.md e CLAUDE.md recebem somente blocos delimitados e atualizáveis.

- O registro fica em `<projeto>/specsfy/skills-lock.json`.
- Cada skill gerenciada registra um fingerprint de conteúdo no lock.
- Uma execução repetida sobre a mesma versão não altera arquivos nem o lock.
- Uma skill gerenciada e intacta pode receber uma versão nova sem `--force`.
- Alterações locais fazem a atualização e a remoção serem recusadas. Use `--force` somente depois de confirmar que a versão customizada pode ser descartada.
- Downloads dos catálogos usam checkout temporário, e `skills add` faz a instalação no projeto.
- `specsfy skills remove <nome>` remove somente o nome explícito e preserva conteúdo local divergente.
- O CLI recusa a raiz reconhecida do workspace `promovaweb/specsfy`.

# Informações permanentes do projeto

O Specsify separa a descrição durável do sistema das especificações de cada mudança. O arquivo `PROJECT.md` explica a finalidade da aplicação, enquanto quatro documentos em `.specsify/` registram a stack, as instruções confirmadas, a persistência observada no código e os pacotes instalados.

Execute `$specsify-setup` depois de instalar o framework ou quando precisar verificar os quatro contextos iniciais. A skill detecta Laravel, Next.js e Astro pelos manifests e sugere o modelo correspondente. `$specsify-documentator` acrescenta e atualiza `PACKAGES.md`. Juntas, as skills mantêm esta estrutura:

```
<projeto>/
├── PROJECT.md
└── .specsify/
    ├── STACK.md
    ├── RULES.md
    ├── DATABASE.md
    └── PACKAGES.md
```

O setup também reserva blocos delimitados para as diretrizes do Specsify em `AGENTS.md` e `CLAUDE.md`. Conteúdo fora desses blocos pertence ao usuário e é preservado. A referência publicável das diretrizes vive em `specsify-setup`.

| ARQUIVO                            | CONTEÚDO                              | SKILL MANTENEDORA                           |
|------------------------------------|---------------------------------------|---------------------------------------------|
| <code>PROJECT.md</code>            | finalidade e capacidades              | <code>\$specsify-setup</code> cria o modelo |
| <code>.specsify/STACK.md</code>    | stack e evidências                    | <code>\$specsify-aux-stack</code>           |
| <code>.specsify/RULES.md</code>    | regras explícitas confirmadas         | <code>\$specsify-aux-rules</code>           |
| <code>.specsify/DATABASE.md</code> | persistência e relações               | <code>\$specsify-aux-database</code>        |
| <code>.specsify/PACKAGES.md</code> | pacotes npm e Composer com finalidade | <code>\$specsify-documentator</code>        |

Os quatro modelos ficam em `.specsify/templates/Project.md`, `Stack.md`, `Rules.md` e `Database.md`, junto dos demais templates do framework. Para personalizar um deles, mantenha o mesmo nome em `.specsify/templates/custom/`; essa cópia tem precedência e não é alterada pelo CLI.

Execute `$specsify-aux-stack` após alterar frameworks, runtimes, ferramentas estruturais ou persistência. Execute `$specsify-aux-database` sempre que criar ou alterar banco, schema, tabela, coleção, model persistente, campo, relação, índice ou migration. Use `$specsify-aux-rules` para formular e acrescentar uma regra confirmada sem duplicar ou apagar regras anteriores.

## Monitoramento durante mudanças

O monitor é executado pelas skills no início e no fim de uma mudança. Ele lê os arquivos staged, unstaged e untracked do Git para descobrir qual documento precisa ser revisto, mas não permanece como daemon em segundo plano:

```
python3 -B .agents/skills/specsfy-setup/scripts/monitor_context.py \
  --project . --check
```

| SINAL OBSERVADO                    | OBRIGAÇÃO                                                                                     |
|------------------------------------|-----------------------------------------------------------------------------------------------|
| manifest, lockfile ou configuração | <code>\$specsify-aux-stack</code> revisa <code>STACK.md</code>                                |
| manifest ou lockfile npm/Composer  | <code>\$specsify-documentator</code> reconstrói <code>PACKAGES.md</code> e <code>docs/</code> |
| schema, model ou migration         | <code>\$specsify-aux-database</code> revisa <code>DATABASE.md</code>                          |
| código da aplicação                | revisar <code>PROJECT.md</code>                                                               |
| aplicação ou persistência          | <code>\$specsify-documentator</code> reconstrói <code>docs/</code> e <code>PACKAGES.md</code> |
| instrução ou convenção             | <code>\$specsify-aux-rules</code> revisa <code>RULES.md</code>                                |

`PENDING` impede a conclusão da tarefa e do Delivery Gate. Quando uma mudança de aplicação não altera história, finalidade, capacidades ou limites, registre essa avaliação na evidência da tarefa e execute novamente com `--acknowledge-project-no-change`. Para uma revisão de regras sem regra nova, use `--acknowledge-rules-no-change` somente depois de registrar a justificativa. Veja a topologia e o `--check` no guia de [documentação técnica do sistema](#).

## Preservação e atualização

- O setup cria um arquivo de informações somente quando ele ainda não existe.
- As auxiliares atualizam apenas blocos detectados delimitados e preservam seções humanas.
- Uma nova varredura nunca autoriza remover silenciosamente uma definição humana.
- `DATABASE.md` usa tabelas Markdown para facilitar mapeamento e comparação.

- `PACKAGES.md` deriva de manifests, lockfiles e metadados locais, preservando texto humano fora do bloco gerado.
- Valores sensíveis não são lidos nem registrados. Cite somente nomes seguros de variáveis e caminhos das fontes.

O estado implementado é comprovado pelas fontes do próprio projeto. Os manifests e lockfiles mostram a stack, enquanto schemas e migrations mostram a persistência. Os cinco documentos resumem essas evidências e as definições humanas que precisam permanecer entre mudanças. Eles não substituem a `spec.md`, não registram gates e nunca devem copiar segredos, valores de `.env` ou registros de produção.

# Documentação técnica do sistema

`$specsify-documentator` reconstrói a visão técnica de uma aplicação em `<projeto>/docs/` e o inventário de dependências em `<projeto>/specsify/PACKAGES.md`. Esses arquivos explicam o código e os pacotes do projeto consumidor e não se confundem com a documentação oficial da metodologia Specsify.

Execute `$specsify-documentator` livremente para documentar um sistema legado ou atualizar sua visão técnica. Depois de cada tarefa de código concluída por `$specsify-07-implement`, a transição para o documentador é obrigatória. A implementação só continua quando `docs/` representar o código atual.

A skill lê o código completo e as fontes que descrevem a aplicação. Isso inclui os manifests, as migrations, as rotas e os testes, além das informações permanentes do projeto. Cada execução reconstrói blocos delimitados nos seguintes arquivos e preserva o texto humano externo:

| ARQUIVO NO CONSUMIDOR              | CONTEÚDO                                                     |
|------------------------------------|--------------------------------------------------------------|
| <code>docs/README.md</code>        | portal e ordem de leitura                                    |
| <code>docs/architecture.md</code>  | componentes, dependências e UML Mermaid                      |
| <code>docs/application.md</code>   | módulos e implementações observadas                          |
| <code>docs/database.md</code>      | entidades, campos, relações e <code>erDiagram</code>         |
| <code>docs/flows.md</code>         | rotas, <code>flowchart</code> e <code>sequenceDiagram</code> |
| <code>docs/testing.md</code>       | runners, comandos, inventário e resumo                       |
| <code>docs/frontend.md</code>      | views, páginas, componentes, React e Tailwind                |
| <code>docs/packages.md</code>      | runtime, framework, nativos, integrados e terceiros          |
| <code>docs/integrations.md</code>  | serviços externos e nomes de configuração                    |
| <code>docs/decisions.md</code>     | escolhas explícitas e suas fontes                            |
| <code>.specsify/PACKAGES.md</code> | pacotes npm e Composer, versão, finalidade e fonte           |

Em Laravel, o inventário acompanha a requisição pelas rotas, controllers e services, relaciona Eloquent e migrations e registra os testes Pest ou PHPUnit. Em projetos Node, Next.js, React ou Astro, a documentação mostra páginas, endpoints, componentes e scripts, além do runner observado no repositório.

Cada pacote recebe a versão e a referência do repositório no GitHub. Quando essa origem não puder ser confirmada localmente, a documentação publica uma busca identificada como tal, em vez de inventar uma URL.

O arquivo `.specsify/PACKAGES.md` percorre todos os manifests npm e Composer do projeto e inclui também as dependências transitivas registradas em `package-lock.json` e `composer.lock`. A finalidade vem da descrição presente no lockfile, no pacote instalado ou no catálogo conhecido do documentador. Quando nenhuma dessas fontes existir, o arquivo declara que a finalidade não foi descrita nos metadados locais.

Depois da reconstrução, a própria skill executa o modo `--check`. O comando compara os blocos gerados com o estado atual e falha quando `docs/` está desatualizado:

```
python3 -B .agents/skills/specsify-documentator/scripts/build_documentation.py \
  --project . --check
```

O monitor do setup também retorna `PENDING` quando o código da aplicação, a persistência ou as dependências mudaram sem uma nova reconstrução de `docs/`. Mudanças em manifests ou lockfiles exigem ainda a atualização de `.specsify/PACKAGES.md`. Esse estado impede a conclusão da tarefa e do Delivery Gate.

O código, os testes, os manifests, os schemas e as migrations comprovam o estado implementado. A spec governa o comportamento da mudança, enquanto `PROJECT.md` e `.specsify/` preservam informações válidas para o sistema inteiro. Os arquivos em `<projeto>/docs/` podem ser reconstruídos dessas fontes e não devem copiar segredos, valores de ambiente, registros de produção ou código integral.

# Atualizar uma especificação

Quando uma entrega já definida precisar mudar, use `specsfy-update-spec`. A skill incorpora a nova instrução na mesma `spec.md` e reconcilia as partes afetadas sem exigir que você escolha atos ou gates manualmente.

## Descreva o que mudou

Durante ou depois da implementação, identifique a `spec.md` e diga diretamente o que deve mudar. Você pode adicionar, remover, corrigir ou mudar uma regra. A skill usará esse arquivo para encontrar os requisitos, testes e tarefas afetados:

```
Use $specsfy-update-spec em
specs/specs/0001-pagina-boas-vindas/spec.md:
esqueci de pedir que o nome tenha no máximo 80 caracteres.
```

Se a spec já estiver clara na conversa, você pode usar uma instrução natural sem mencionar atos ou gates:

```
Quero adicionar uma regra à especificação atual.
Remova o comportamento de visitante.
Corrija o que acontece quando o nome estiver vazio.
Mude esta entrega para exigir autenticação.
```

A skill preserva a instrução, compara-a com a spec existente e informa primeiro o resultado prático. Você não precisa escolher manualmente qual gate ou etapa reabrir.

## O que acontece

| TIPO DE MUDANÇA                        | TRATAMENTO                           |
|----------------------------------------|--------------------------------------|
| correção interna sem efeito observável | mantém a spec                        |
| esclarecimento sem novo significado    | ajusta o texto e preserva os gates   |
| mudança de comportamento ou aceite     | atualiza a spec e reabre o Ato I     |
| mudança de solução, tarefas ou testes  | atualiza a spec e reabre o Ato II    |
| capacidade independente                | encaminha para backlog ou nova spec  |
| definição material ausente             | pergunta e retoma depois da resposta |

Quando a mudança pertence à spec atual, a skill atualiza o arquivo, encaminha as etapas invalidadas e retorna à implementação somente depois das novas provas:

```
mudança posterior → update-spec → validate ou tasks → TDD/BDD → implement
                        ↘ backlog, se faltar uma definição
```

Testes e tarefas que continuam válidos são preservados. Provas dependentes do comportamento anterior voltam a ficar pendentes. A implementação só é retomada depois de existir novo RED válido e os gates necessários voltarem a `Passed`.

## Resultado esperado

- a nova instrução está incorporada à única `spec.md` normativa.
- IDs e definições ainda válidos foram preservados.
- os gates afetados foram reabertos.
- validação, tarefas e testes foram reconciliados.
- a etapa que recebeu o pedido foi retomada automaticamente.

## Limites

- não cria `change-request.md`, `plan.md`, `tasks.md` ou outra fonte paralela.
- não transforma uma capacidade independente em aumento silencioso de escopo.
- não inventa uma definição material.
- não altera código antes de atualizar a spec e preparar novo RED.
- handoffs automáticos não autorizam deploy, publicação ou ação destrutiva.

## Quando usar outra skill

- criar a primeira versão de uma spec.
- capturar uma ideia ainda superficial.
- editar código quando a spec continua correta.

Depois da atualização, consulte [specsfy-progress](#) para conferir o estado geral. A spec atualizada permanece como única fonte normativa do projeto.

# Skills especialistas

As skills base conduzem a metodologia. As `specsify-specialist-*` acrescentam orientação para a tecnologia encontrada no projeto, como Laravel, Astro, Next.js, Postgres ou Redis. Assim, você instala somente o conhecimento técnico usado pela aplicação.

## Detectar e instalar

O comando `detect` lê o projeto e mostra recomendações sem instalar arquivos. O catálogo local só deve ser alterado depois que você revisar os nomes retornados:

```
specsify skills detect
```

Depois da revisão, informe ao comando `add` apenas os especialistas usados pela aplicação. O CLI registra cada instalação no `skills-lock.json`, permitindo conferir depois quais arquivos são gerenciados:

```
specsify skills add \  
  specsify-specialist-laravel \  
  specsify-specialist-postgres \  
  specsify-specialist-redis
```

As skills base podem sugerir um especialista. Quando ele já estiver instalado, a transição é anunciada e continua na mesma conversa. Quando estiver ausente, o agente precisa de autorização específica para instalar. A transição entre skills nunca instala o catálogo inteiro automaticamente.

## Catálogo por domínio

| DOMÍNIO         | ESPECIALISTAS                                         |
|-----------------|-------------------------------------------------------|
| backend e dados | Laravel, Supabase, Postgres, Redis e APIs web         |
| frontend        | React, Astro, Next.js, TypeScript e Tailwind CSS      |
| interface       | shadcn/ui, UI, UX e acessibilidade web                |
| plataforma      | Docker, Docker Swarm, Ansible e engenharia de entrega |
| qualidade       | segurança, observabilidade e performance              |
| design técnico  | arquitetura e modelagem de domínio                    |

| DOMÍNIO    | ESPECIALISTAS                                                  |
|------------|----------------------------------------------------------------|
| engenharia | code review, debugging, prototipação, pesquisa e conflitos Git |

Cada especialista explica o fluxo de trabalho e as validações próprias da sua tecnologia. Uma mudança em Eloquent pode combinar Laravel e Postgres, por exemplo, mas uma alteração isolada em uma página Astro não precisa carregar orientações de todo o catálogo.

## Relação com as bases

- `specsify-02-backlog` identifica a necessidade.
- `specsify-03-specify` registra requisitos e atributos de qualidade.
- `specsify-04-validate` seleciona lentes de revisão.
- `specsify-05-tasks` usa checklists técnicos para decompor trabalho.
- `specsify-06-tdd-bdd` preserva RED/GREEN.
- `specsify-07-implement` executa de acordo com a stack.
- `specsify-update-spec` revisa requisitos e atributos de qualidade afetados por uma mudança posterior.
- `specsify-progress` identifica a orientação técnica aplicável e executa a transição automática.

A presença de um especialista não aprova gates. O Plan Gate ainda exige um RED válido, e o Delivery Gate depende dos testes e das evidências registradas na spec.

# Uso avançado do Specsify

---

Este guia reúne recursos usados depois da primeira spec: seleção de especialistas, progresso em JSON, atualização visual e retomada de uma mudança posterior. Para acompanhar os exemplos, conclua o [primeiro projeto](#), mantenha o CLI e o framework instalados e execute os comandos na raiz do projeto consumidor.

## Detecte e instale orientação técnica

Comece com `skills detect` para ler as recomendações do catálogo sem alterar o projeto:

```
specsify skills detect --project .
```

Quando todas as recomendações forem aplicáveis, `--detected` instala o framework e os especialistas em uma única execução. O `skills-lock.json` registra os arquivos publicados:

```
specsify install --project . --detected
```

Quando o catálogo listar tecnologias que não pertencem à aplicação, repita `--specialist` somente com os nomes confirmados. Assim, o instalador publica as bases e ignora as recomendações que não foram escolhidas:

```
specsify install --project . \  
  --specialist specsify-specialist-laravel \  
  --specialist specsify-specialist-postgres
```

Em um projeto já preparado, use `skills add` para acrescentar somente os especialistas escolhidos:

```
specsify skills add specsify-specialist-laravel --project .
```

A detecção usa manifests, dependências e arquivos reconhecidos pelo catálogo. O comando de instalação só deve ser executado depois da revisão. Os especialistas orientam escolhas técnicas, mas não criam specs nem aprovam gates.

## Automatize a leitura de progresso

```
specsify progress --project . --json  
specsify progress --project . --watch --interval 0.5 --json
```

O JSON contém `summary` e `specs`. Com `--watch`, um snapshot novo aparece somente quando o conteúdo das specs muda. Painéis podem consumir essa saída para leitura, mas os gates continuam sendo editados apenas na `spec.md` pela skill responsável.

## Ajuste a atualização visual

```
specsfy config show --project .  
specsfy config set --project . --watch-interval 0.5
```

A configuração vive em `<projeto>/specsfy/config.json` e preserva chaves desconhecidas. Consulte todos os recursos no [guia do CLI](#).

## Atualize sem perder customizações

```
specsfy skills update --project .
```

O CLI usa fingerprints para distinguir conteúdo gerenciado intacto de customização local. Uma diferença local faz a atualização ou a remoção ser recusada. Use `--force` somente depois de revisar a comparação e confirmar que o conteúdo protegido pode ser descartado.

Quando o CLI foi instalado pelo npm, o comando abaixo atualiza o pacote global:

```
npm update --global @promovaweb/specsfy
```

Quando a instalação usa o executável Node oficial, substitua-o pelo download mais recente e restaure a permissão:

```
curl -fL get.specsfy.dev -o "$HOME/.local/bin/specsfy"  
chmod +x "$HOME/.local/bin/specsfy"
```

## Incorpore uma mudança na spec

Quando lembrar de algo depois da definição, use a entrada explícita:

```
Use $specsfy-update-spec em  
specs/specs/<NNNN>-<slug>/spec.md:  
quero adicionar, remover, corrigir ou mudar esta especificação.
```

Quando um requisito observável muda, a skill reabre a definição. O Definition, o Plan e o Delivery Gate precisam de evidência nova. Quando apenas o plano técnico muda, ela reabre o Ato II e o Ato III. A spec alterada invalida as provas que dependiam da versão anterior e preserva tarefas e evidências ainda compatíveis.

As skills fazem transições e retomadas na mesma conversa. Esse handoff não amplia autorização para instalar, publicar, fazer deploy ou executar ação destrutiva.

Consulte [Atualizar uma especificação](#) para a classificação completa e exemplos em linguagem comum.

## Limites

- `--force` pode descartar customizações protegidas.
- `--detected` depende do catálogo e do estado observado no projeto.
- A TUI e o progresso projetam estado, mas não substituem a spec.
- o especialista não substitui a confirmação da versão, das convenções e das falhas possíveis no projeto.

O resultado esperado é um projeto com especialistas escolhidos de forma explícita, automação somente de leitura e gates coerentes com a versão atual da spec.

# Usar Specsify com Laravel

---

`$specsify-specialist-laravel` acrescenta verificações próprias do Laravel ao fluxo do Specsify. A spec continua governando a mudança, e o especialista segue as convenções e a versão comprovadas pelo projeto.

## Confirmar a detecção

O catálogo detecta Laravel por `artisan`, `composer.json` ou pela dependência `laravel/framework`. Confirme a versão e as extensões em `composer.json` e `composer.lock`. Uma API só deve orientar a implementação quando existir na versão usada pela aplicação.

## Instalação

Na raiz do projeto, `--detected` instala o framework e o especialista quando o catálogo reconhece Laravel:

```
specsify install --project . --detected
```

Para revisar a recomendação sem instalar arquivos, use `skills detect`. A saída deve incluir `specsify-specialist-laravel` quando `artisan` ou a dependência do framework for encontrada:

```
specsify skills detect --project .
```

Quando o nome já estiver confirmado, `skills add` instala somente o especialista Laravel e registra os arquivos gerenciados:

```
specsify skills add specsify-specialist-laravel --project .
```

## Aplicar na spec

1. Capture ou promova a ideia pelo [primeiro projeto](#).
2. Peça ao agente para usar `$specsify-specialist-laravel` na fatia ativa.
3. Confirme a versão, extensões, convenções locais e o caminho da requisição.
4. Na definição e no plano, registre autorização e validação. Quando a mudança alcançar persistência ou execução assíncrona, inclua transações, idempotência, filas, falhas e migrations.
5. Derive testes para caminho feliz, autorização, validação, efeitos e falhas.

6. Implemente controllers finos e mantenha as regras na camada já adotada pelo projeto. Inspeccione as consultas e o N+1 quando a quantidade de relações puder aumentar o tempo da resposta.
7. Execute os checks existentes. Em Laravel com Pest, o CLI oferece:

```
specsify test --project .
```

O CLI detecta `artisan` e `pestphp/pest`, chama `php artisan test` e preserva o exit code. Ele não recebe uma string arbitrária de shell.

## O que o especialista acrescenta

- contratos HTTP, Form Requests, policies, resources e bindings.
- Eloquent, eager loading, casts e transações conscientes.
- jobs idempotentes, tentativas, backoff e tratamento de falha.
- migrations compatíveis com volume, locks, rollback e deploy misto.
- verificação de queues, scheduler, cache, configuração e ambiente.

## Resultado esperado

A spec continua sendo a fonte normativa, enquanto os testes e a implementação consideram as falhas possíveis do Laravel observado no projeto e na versão instalada.

## Limites

- não aplique a skill a PHP sem Laravel.
- não presuma APIs pela versão mais recente da documentação.
- não execute migration, deploy ou comando operacional sem autorização.
- não confie apenas na validação ou autorização da interface.

Não use esse especialista para PHP sem Laravel nem para fixar uma versão que o projeto não comprova. O código, `composer.json`, `composer.lock`, os testes e a configuração local permanecem como evidência do estado da aplicação.

# Usar Specsify com Astro

---

`$specsify-specialist-astro` acrescenta ao fluxo do Specsify as escolhas de renderização, conteúdo, ilhas e performance próprias de um site Astro. A skill usa a versão e a configuração do projeto, sem presumir o adapter ou o modo de saída.

## Confirmar a detecção

O catálogo detecta Astro pela dependência `astro` em `package.json` ou pelos arquivos `astro.config.mjs` e `astro.config.ts`. Descubra no projeto a versão, o output mode, o adapter, as integrações e as fontes de conteúdo.

## Instalação

```
specsify skills detect --project .  
specsify skills add specsify-specialist-astro --project .
```

Quando todas as recomendações exibidas forem aplicáveis, `--detected` instala o framework e os especialistas em uma única execução. Confira depois se `specsify-specialist-astro` aparece no catálogo instalado:

```
specsify install --project . --detected
```

## Aplicar na spec

1. Conduza a ideia até a spec conforme o [primeiro projeto](#).
2. Peça ao agente para usar `$specsify-specialist-astro` na fatia ativa.
3. Classifique cada rota afetada como estática, sob demanda ou endpoint.
4. Mantenha HTML estático por padrão e escolha uma diretiva `client:*` somente para a interação que precisa de hidratação.
5. Modele conteúdo com schemas e relações explícitas. Defina cache, headers, imagens e metadados quando aplicáveis.
6. Crie testes derivados do Gherkin para rotas, conteúdo inválido e comportamento hidratado.
7. Execute `astro check`, a suíte do projeto, o build de produção e o preview no runtime do adapter disponível no projeto.

## O que o especialista acrescenta

- escolha explícita entre static, on-demand, island e endpoint.
- proteção contra transporte de dados sensíveis para ilhas.
- validação de slugs, relações e conteúdo.
- semântica, canonical, sitemap e dados estruturados.
- inspeção de payload cliente, Core Web Vitals e compatibilidade do adapter.

## Resultado esperado

A fatia preserva a fonte normativa Specsify e torna observáveis a estratégia de renderização, a hidratação necessária, os contratos de conteúdo e a validação no runtime alvo.

## Limites

- não escolha SSR sem necessidade de sessão, personalização ou frescor.
- não suponha APIs de Node em adapters edge.
- não valide apenas no servidor de desenvolvimento.
- confirme scripts e comandos disponíveis no `package.json`.

Não use esse especialista para prescrever um adapter ou modo de saída sem evidência. A versão, o adapter, os scripts e as integrações são comprovados pelos manifests, lockfiles e pela configuração do projeto consumidor.

# Usar Specsify com Next.js

---

`$specsify-specialist-nextjs` acrescenta ao fluxo do Specsify as escolhas de Server e Client Components, cache, mutations, rotas e deploy próprias do Next.js. A skill confirma o router e a versão porque os padrões de cache mudam entre gerações do framework.

## Confirmar a detecção

O catálogo detecta Next.js pela dependência `next` em `package.json` ou por `next.config.js`, `next.config.mjs` e `next.config.ts`. Confirme a versão e o router. O runtime, o destino de deploy e as flags ativas também precisam ser registrados no plano.

## Instalação

```
specsify skills detect --project .  
specsify skills add specsify-specialist-nextjs --project .
```

Quando todas as recomendações forem aplicáveis, `--detected` instala as bases e os especialistas em uma única execução. Confira depois se `specsify-specialist-nextjs` aparece no catálogo instalado:

```
specsify install --project . --detected
```

## Aplicar na spec

1. Conduza a ideia até a spec pelo [primeiro projeto](#).
2. Peça ao agente para usar `$specsify-specialist-nextjs` na fatia ativa.
3. Mapeie a rota, o layout e os estados `loading`, `error` e `not-found`. Registre também o limite entre o código do servidor e a interação no navegador.
4. No App Router, mantenha componentes no servidor por padrão e mova ao cliente apenas o componente que requer estado, eventos ou APIs do navegador.
5. Defina cache, revalidation, tags e comportamento dinâmico explicitamente.
6. Trate Server Actions e Route Handlers como superfícies públicas: valide autenticação, autorização e entrada em cada mutation.
7. Derive testes para estados, redirects, autorização, invalidação e isolamento de dados entre usuários.
8. Execute lint, typecheck, testes, build e runtime de produção conforme os scripts do `package.json`.

## O que o especialista acrescenta

- controle do limite entre Server e Client Components.
- prevenção de segredos e módulos server-only no bundle cliente.
- análise de waterfalls, streaming e recuperação de erro.
- cache como contrato compatível com a versão instalada.
- metadata, assets, bundle, imagens, fontes e Web Vitals.

## Resultado esperado

As escolhas de renderização, cache e segurança ficam rastreadas pela spec e provadas por testes e build na versão e no router realmente usados.

## Limites

- não presuma App Router ou semântica de cache sem confirmar a versão.
- não mova uma árvore inteira ao cliente por conveniência.
- não use middleware para lógica longa ou incompatível com o runtime.
- não assuma comportamento específico do host sem documentá-lo.

Não use esse especialista para escolher App Router, Pages Router ou runtime sem evidência. O código, `package.json`, o lockfile e a configuração comprovam o router, a versão, o runtime e os scripts disponíveis no projeto consumidor.

# Créditos do Specsfy

---

Esta página registra a autoria pública do Specsfy e aponta as fontes usadas para confirmar contribuições, identidade visual e componentes relacionados. O histórico Git preserva a autoria detalhada de cada mudança.

## Projeto e manutenção

Specsfy é um projeto da Promovaweb, criado e mantido por **Luiz Eduardo Oliveira Fonseca** com a comunidade. O contato público do projeto é [contato@promovaweb.com](mailto:contato@promovaweb.com).

## Comunidade

Contribuições em documentação, código, testes e pesquisa fazem parte da evolução do projeto. O histórico do monorepo preserva a autoria de cada arquivo e commit, por isso este guia não mantém uma lista manual que ficaria desatualizada.

## Identidade

Logos, cores, tipografia, voz, acessibilidade e regras de aplicação pertencem ao diretório `brand/`. Use os ativos e orientações desse diretório ao apresentar o projeto.

## Componentes relacionados

O instalador do Specsfy delega a instalação de skills ao projeto `vercel-labs/skills`. O CLI pode usar o executável `skills` ou o fallback por `npx`, conforme documentado no [guia de instalação](#).

## Inspirações e fontes

O Specsfy desenvolve um contrato executável próprio e reconhece estas referências como inspiração:

- GitHub Spec Kit: aplicação de specification-driven development em etapas próximas ao código.
- OpenSpec: especificações e mudanças mantidas no repositório como um acordo leve entre o desenvolvedor responsável e o agente.
- Categorias, de Aristóteles: referência filosófica para classificar objetos, atributos, relações e estados antes de formular afirmações sobre eles.

As referências não são dependências do Specsfy e não tornam os métodos equivalentes. As skills, os templates, os validadores e os testes deste repositório continuam sendo as fontes do comportamento executável.

## Como contribuir

Localize primeiro o diretório responsável no quadro dos módulos, leia as instruções locais e preserve os limites de cada repositório Git. A documentação oficial fica neste repositório. O comportamento executável e os testes permanecem no módulo que implementa cada recurso.

O histórico e os arquivos de licença de cada repositório comprovam autoria e licenciamento. Quando uma licença não estiver declarada, não presuma seus termos. A identidade oficial do Specsfy permanece em `brand/`.